Software Engineering Foundations

ENTERPRISE MOBILE APPLICATION DEVELOPMENT BOOTCAMP

PHASE 01

Software Engineering Foundations for Mobile

Phase 01 - Three Modules at a Glance

Module	Schedule	Core Skills
Module 1 Software Engineering Principles	Day 1 & 2	SDLC · Agile/Scrum · Platform strategy · SOLID (all 5) · Clean code · Technical debt · User stories
Module 2 Source Control and Team Development	Day 3 & 4	Git internals · Core operations · GitFlow / TBD / GitHub Flow · Conventional Commits · PRs · Code review · Conflicts · Advanced Git
Module 3 Problem Solving and Engineering Thinking	Day 5	Decomposition · Technical communication · Estimation · 7-step debugging · Root cause analysis · Architecture whiteboarding

Modern Software Engineering Principles

Module 1

The Software Development Lifecycle

THE PROBLEM

Why can't we just start coding?

Every project that skips planning pays for it in rework, missed requirements, and production defects.

The Seven SDLC Phases

Phase	What Happens	What Gets Skipped = What Goes Wrong
Planning	Scope, resources, risks, feasibility	Scope creep, missed deadlines, wrong team size
Requirements	What the software must do (functional) and how (NFRs)	Building the wrong thing. Rework.
Design	Architecture, API contracts, data models, UI flows	Incompatible components. Integration surprises.
Development	Code, review, integrate	Without prior phases: chaos
Testing	Correctness, performance, security, usability	Production defects. Loss of user trust.
Deployment	Release through a controlled process, CI/CD	Unreproducible deployments. Rollback failures.
Maintenance	Monitor, fix, iterate - 60-80% of lifecycle cost	Technical debt accumulation. Slow teams.

SDLC Models - Which Fits Your Project?

- Waterfall: sequential, each phase completes before the next - rigid, but right for fixed regulatory specs
- Iterative: phases repeat in cycles - earlier feedback, some flexibility
- Agile / Scrum: two-week sprints, working software every cycle - dominant model for enterprise mobile
- Spiral: risk-driven iterations - used in safety-critical systems (medical, aerospace)
  - Is a meta-model, combining features of Waterfall, Agile, and Iterative
- This program uses Agile/Scrum throughout - the model you will use in every Phase lab

Which SDLC Phase Gets Skipped Most?

Skill Check

Your Turn

Which phase do you think teams skip most often in real projects? What happens as a result?

Agile and the Scrum Framework

THE AGILE MANIFESTO VALUES (2001)

Individuals and interactions over processes and tools.
Working software over comprehensive documentation.
Customer collaboration over contract negotiation.
Responding to change over following a plan.

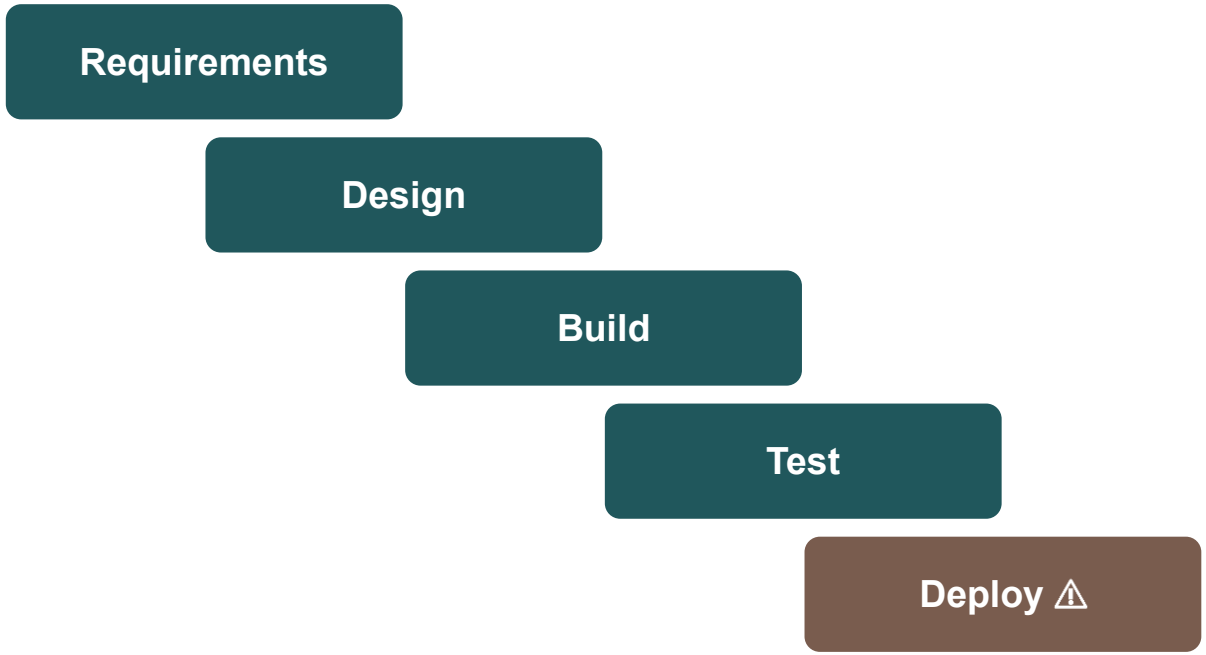
The items on the right have value - we just value the items on the left more.

The 12 Agile Principles - Operational Meaning

#	Principle	What It Means in Mobile Engineering
1	Early and continuous delivery of working software	Builds to TestFlight/Firebase every Sprint - not demos, not prototypes
9	Technical excellence and good design enhance agility	SOLID, clean code, tests - not opposed to speed, they enable it
10	Simplicity - maximizing the work NOT done	Build what's needed now. Over-engineering is as dangerous as under-engineering
12	Reflect and adjust at regular intervals	The retrospective operationalizes this - it's not optional

Waterfall Cascades. Scrum Loops.

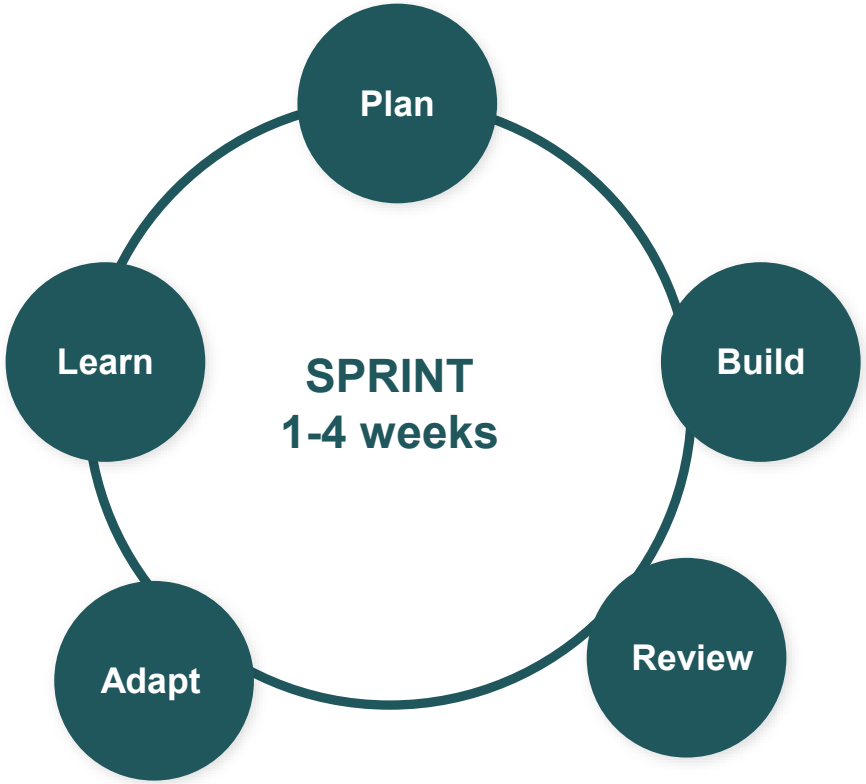
WATERFALL - value arrives at the end



⚠ ***Feedback arrives here - months later***

Change is expensive. Risk is discovered late. Stakeholders see the product once.

SCRUM - value arrives every Sprint



✓ ***Working product + feedback every cycle***

Change is cheap. Risk surfaces early. Stakeholders see real results continuously.

Scrum Roles - What They Are and Are Not

Role	Core Responsibility	What They Are NOT
Product Owner	Owns and orders the backlog. Single point of contact for business requirements.	Not a requirements gatherer. Not someone who can be unavailable for days.
Scrum Master	Serves the team. Removes impediments. Protects the process. Coaches.	Not a project manager. Not a task-assigner. Not a secretary.
Developers	Create the increment. Collectively accountable for quality and delivery.	Not just coders - everyone who contributes to the increment.

Three Accountabilities. One Team. Zero Hierarchy.

Product Owner

Owns VALUE

- Orders the Product Backlog
- Makes prioritization decisions
- Carries the product vision
- One person - not a committee

Scrum Master

Owns EFFECTIVENESS

- Coaches the team on Scrum
- Removes impediments
- Shields team from interference
- Leads by serving - not directing

Developers

Own the HOW

- Everyone building the Increment
- Self-managing - no task assignment
- Collectively own quality (DoD)
- Cross-functional by design

The Product Owner: Value Decision-Maker

The PO is accountable for…

- Maximizing the value of the product
- Owning and ordering the Product Backlog
- Deciding what to build next - by value and risk
- Communicating the product vision and goal
- Being available to the team for fast answers

The PO is not…

- A requirements writer who hands off a spec
- A committee or a rotating seat
- A task assigner directing the Developers
- A proxy who must escalate every decision
- Optional - an absent PO starves the team

The Scrum Master: Coaching, Not Policing

Serves the team

Removes impediments blocking flow; creates the conditions for focus

Serves the Product Owner

Helps with backlog techniques and stakeholder collaboration

Serves the organization

Coaches leadership on how to work with - not over - Scrum teams

Never: assigns work · manages people · reports team status upward

When the SM starts doing these, self-management dies - and Scrum with it.

"Who does status reporting, then?"

Nobody - because the artifacts and events ARE the status. The Sprint Review shows working product. The backlog shows what's done and what's next. Transparency by default, not by report.

Developers: Everyone Who Builds the Increment

“Developer” is not a job title in Scrum - it's anyone whose work creates the Increment: engineers, testers, designers, analysts, writers, data scientists.

Cross-functional accountability

The team collectively owns delivery - no “not my job” inside the Sprint

They create the Sprint plan

The PO says what and why. The Developers decide how - and estimate it

Quality lives with the team

The Definition of Done is enforced by the people doing the work

Self-management, in reality

Not the absence of management - management that lives inside the team rather than above it

Three Artifacts, Three Commitments

WHAT + WHY

Product Backlog

The single, ordered list of everything that might improve the product. Never complete - it evolves as the team learns.

Commitment: Product Goal

THIS SPRINT

Sprint Backlog

The Sprint Goal, the selected items, and the team's plan. Owned by the Developers and updated daily.

Commitment: Sprint Goal

WHAT'S REAL

Increment

All completed work - usable, meeting the quality bar. If it doesn't meet the DoD, it isn't done.

Commitment: Definition of Done

The Product Backlog: What Belongs, What Doesn't

Belongs in the Product Backlog

- Features and user stories
- Defects and technical debt
- Compliance and regulatory items
- Research spikes and experiments
- Improvements surfaced by Retros

Doesn't belong

- Task-level breakdowns (that's the Sprint Backlog)
- Items no one intends to ever do
- Duplicate wish lists from every stakeholder
- Hidden side-channel work

Order for value and risk - not for politics.

High value + high risk items rise to the top: learn the hard things early, while change is still cheap.

Sprint Backlog

Sprint Backlog

A forecast, not a contract

“We believe we can do this” - updated daily as the team learns

Owned entirely by the team

No outside party adds, removes, or verifies items

Transparency, not control

Visible to everyone; managed by no one above the team

Increment - Delivered Working Product

Definition of Done

“Done” is the most dangerous word in delivery. Coded but not tested? Tested but not deployed? Deployed but not accepted?

The DoD is a quality contract:

- One shared standard for “complete”
- Applied to every item, every Sprint
- If it doesn't meet the DoD - it is not done. Period.

Writing User Stories and Acceptance Criteria

The INVEST Criteria - What Makes a Good Story

Letter	Characteristic	What it Means
I	Independent	The story can be developed without depending on other stories
N	Negotiable	The story is a placeholder for conversation, not a detailed contract
V	Valuable	The story delivers value to a user, customer, or stakeholder
E	Estimable	The team understands it well enough to estimate the effort
S	Small	The story is small enough to fit comfortably within a sprint
T	Testable	The team can verify whether the story is done

From BRD Statement to User Story

BRD STATEMENT

“The system shall allow users to filter search results by date range, category, and price.”

USER STORY

As a **customer**, I want to **filter my search results by date range, category, and price**, so that **I can select the right product for my needs.**

The anatomy - and why each part matters

WHO — **As a [type of user]**

Keeps a real human in the frame - not “the system”

WHAT — **I want to [action]**

The capability, stated in user language

WHY — **So that [benefit]**

The value test - if you can't fill this in, ask why it's in the backlog

Writing Acceptance Criteria

- Acceptance criteria define the conditions under which a user story is considered complete
- They should be included in a user story that is ready for development
  - Definition of Ready
- The Gherkin syntax (Given/When/Then) structures them as testable scenarios:
  - Given: The preconditions or context (the world's state before the action)
  - When: The action the user takes or the event that occurs
  - Then: The expected outcome - the system's new state after the action
  - And: Additional conditions chained to any of the above

Why Gherkin?

Gherkin scenarios can be directly automated using Cucumber, XCTest (iOS), or Espresso (Android). A story with complete Given/When/Then scenarios is 80% of the way to an automated acceptance test suite. Writing them forces precision in requirements before any code is written.

Given / When / Then - Acceptance Criteria

Story: As a field technician, I want to log hours offline so that I can work in areas with no connectivity.

Scenario 1: Logging while offline

Given the device has no network connectivity and the user is on the Time Entry screen

When the user enters 4.5 hours for Project Alpha and taps Save

Then the entry is saved to local storage, and the entry appears in the list with a 'Pending Sync' label

Scenario 2: Sync on reconnect

Given the device has 3 entries with 'Pending Sync' status

When the device regains network connectivity

Then the app automatically syncs all pending entries and successfully synced entries update their status to 'Submitted'

Common User Story Anti-Patterns

Anti-Pattern	Example	Problem	Fix
Technical task masquerading as a story	"As a developer, I want to refactor the network layer so that it uses protocols."	No user value. The 'actor' is the developer, not a system user.	Developers are not story actors. Track technical work as tasks or enablers with clear rationale tied to user value.
Missing business value	"As a manager, I want to see a dashboard."	No 'so that' clause. Why do they want a dashboard? What decision does it enable?	"...so that I can identify which employees have not submitted timesheets before the Friday deadline."
Feature-as-story (too large)	"As a user, I want to manage my account."	Unestimable. Includes update email, change password, update profile photo, deactivate account, set notification preferences - each a separate story.	Split into individual capabilities, each independently valuable.
Prescribes implementation	"As a user, I want a modal dialog that appears when I tap the + button."	Specifies the solution, not the need. Design is not the story author's domain.	"As a user, I want to add a new time entry from the timesheet screen." (Let designers and engineers decide the interaction pattern.)

User Story Writing Workshop

Skill Check

- Rewrite the following Business Requirements as User Stories, complete with acceptance criteria:
- Feature request: 'The app should let managers see who hasn't submitted their timesheet yet.'
- Feature request: 'We need the app to handle when field technicians work in areas with no cell service.'
- Feature request: 'Employees should be able to correct a timesheet after submission if they made a mistake.'

Scrum Ceremonies - Purpose and Anti-Patterns

Ceremony	Time-box	Purpose	Most Common Anti-Pattern
Sprint Planning	4 hrs (2-wk sprint)	Sprint Goal + Sprint Backlog	Planning without a Sprint Goal - just a list of tasks
Daily Scrum	15 min	Inspect progress toward Sprint Goal	Status report to manager instead of peer sync
Sprint Review	2 hrs (2-wk sprint)	Demo Done work to stakeholders, adapt backlog	Showing not-Done work. No stakeholders present.
Sprint Retrospective	1.5 hrs	Improve process - specific actions with owners	12 items with no owners and no due dates

Five Events, One Continuous Loop

Timeboxes (for a 1-month Sprint)

Sprint Planning	8 hrs max
Daily Scrum	15 min - always
Sprint Review	4 hrs max
Retrospective	3 hrs max

Shorter Sprints scale these down proportionally. The timebox is a ceiling, not a target.

Sprint Planning

Sprint Planning answers three questions

WHY **is this Sprint valuable?**

The PO proposes; the team crafts the Sprint Goal together

WHAT **can be done this Sprint?**

Developers pull items from the top of the backlog - they choose the amount

HOW **will the work get done?**

The team plans its own approach. No one plans it for them

The Daily Scrum

The Daily Scrum is NOT a status meeting

15 minutes, daily, for the Developers

Purpose: inspect progress toward the Sprint Goal and adapt the plan

No one reports to anyone - the team coordinates with itself

SM and PO attendance: optional

The 3-question script is a guideline, not a rule - many teams walk the board instead

Each team member answers three questions:

WHAT have you done since yesterday?

WHAT are you planning for today?

WHAT roadblocks are you currently facing?

Sprint Review

Sprint Review

Inspects the PRODUCT

- Working Increment shown to real stakeholders
- Feedback flows into the Product Backlog
- Not a polished demo - an honest inspection
- Replaces the “big reveal” at the end of a Waterfall project

Compare: in Waterfall, stakeholders often see the product for the first time at UAT - when change is most expensive.

Sprint Retrospective

Sprint Retrospective

Inspects the PROCESS

- How did we work together this Sprint?
- What will we improve - concretely - next Sprint?
- The engine of continuous improvement
- Most skipped when pressure is high - exactly when it's most needed

A Retro that produces the same action items every Sprint isn't a Retro. It's a ritual.

Effort Estimation and Planning Poker

Scrum Does Not Require Story Points

The Scrum Guide mentions story points…

0 times

Scrum requires that the team can size work well enough to make a credible Sprint forecast. How is entirely up to the team.

When estimation helps

- Forces conversation that surfaces hidden complexity
- Builds shared understanding before work starts
- Supports credible forecasting for stakeholders

When estimation hurts

- Debating points consumes more time than the work
- Estimates get treated as commitments - then weaponized
- Precision theater: false confidence in unknowable numbers

Common Estimation Techniques

Story Points

Relative sizing (Fibonacci). Measures complexity, not hours.

Best for: Teams wanting effort decoupled from time

Planning Poker

Simultaneous reveal prevents anchoring. Discussion on splits.

Best for: Surfacing disagreement that hides complexity

T-Shirt Sizing

S / M / L / XL. Fast, low-argument, roughly right.

Best for: Early grooming and roadmap-level planning

Right-Sizing

Split everything until it's “a day or two.” Count items.

Best for: Mature teams who want radical simplicity

Raw Time

Hours or days. Familiar and simple.

Best for: Teams new to Scrum or non-software contexts

Throughput

Forecast from historical items-per-Sprint data.

Best for: Stable teams with consistent item sizes

Story Points - What They Mean in Practice

Points	Meaning	Mobile Example
1	Trivial. Completely understood. No unknowns.	Change an error message. Fix a typo. Update a button color.
2	Small, well-understood. One component.	Add a loading spinner to an existing screen. Show one additional field.
3	Moderate. 2-3 components. Straightforward tests.	New form with validation and API call. Filter option on existing list.
5	Significant. Multiple components. Integration testing needed.	New screen from one API endpoint - loading, error, empty, populated states.
8	Complex. Cross-cutting. Significant unknowns.	Push notification handling with deep link. Offline-capable screen.
13	Very complex. Architecture implications. Multiple unknowns.	Offline sync with conflict resolution. Biometric auth on existing flow.
21	Too large. Must be decomposed before committing.	Any 21-point story should be split. If it won't split, spike first.

Planning Poker - Mechanics and Value

- PO reads the story. Team asks clarifying questions. Questions stop when no new information is coming.
- Each team member privately selects their estimate card.
- All cards revealed SIMULTANEOUSLY - this prevents anchoring (the first number heard biases everyone else).
- If estimates are identical or within one Fibonacci value: adopt. Brief discussion.
- If estimates diverge significantly: HIGH estimator explains first, then LOW. Discussion follows.
- Revote after discussion if needed. Typically 1-2 rounds to consensus.
- The most valuable moment: when a 3 and a 13 appear on the table. Don't rush to consensus - the divergence is the information.

7 Estimation Anchors - Questions to Ask Before Voting

Question	Direction if YES
Have we built something similar before?	YES → lower estimate. NO → higher estimate or spike.
Does this require a spike first?	YES → do not estimate this story yet. Spike it first.
Does this touch iOS AND Android?	YES → multiply, don't just add. Platform parity is real cost.
Does this touch multiple services or teams?	YES → add coordination overhead - meetings, waiting, alignment.
Are there security or compliance requirements?	YES → add significant weight. Security review is never free.
Are there performance requirements needing profiling?	YES → add time for measurement and tuning cycles.
Are there accessibility requirements?	YES → add implementation and VoiceOver/TalkBack test time.

Story Point Estimation - Four Stories

Story 1

Add a note field to an existing time entry form.

Story 2

Daily summary push notification for managers.

Story 3

GPS geofencing to auto-detect shift start/end.

Story 4

Configurable overtime threshold per contract type.

Plan your Sprint from this Backlog

Skill Check ✓

#	User Story	Points	Priority	In Sprint 1?
1	As a user, I can log in with SSO credentials	?	Critical	Y / N
2	As a user, I can view my account dashboard	?	High	Y / N
3	As a user, I can update my profile information	?	High	Y / N
4	As an admin, I can assign user roles	?	Medium	Y / N
5	As a user, I can reset my password via email	?	High	Y / N
6	As a user, I can view my transaction history	?	Medium	Y / N
7	As a user, I receive email notifications on order status	?	Medium	Y / N
8	As an admin, I can export user data to CSV	?	Low	Y / N
9	As a user, I can set notification preferences	?	Low	Y / N
10	As a user, I can access help documentation	?	Low	Y / N

Platform Selection Strategy

Native vs. Hybrid vs. Cross-Platform

Approach	Technologies	Pros	Cons	Best For
Native	Swift/Xcode · Kotlin/Android Studio	Max performance · Full API access · Best UX fidelity	Two codebases · Highest cost	Enterprise apps · Complex hardware integration · 5+ year lifespan
Hybrid	Ionic + Capacitor · Cordova	One codebase · Web dev familiarity	Bridge overhead · Limited native APIs · Inconsistent UX	Simple internal tools · Low-interaction apps
Cross-Platform	React Native · Flutter · KMM	Shared logic or UI · Growing ecosystem	Abstraction layers · Platform-specific edge cases	Teams with JS/Dart experience · 3-5 year horizon

Decision Framework

Question	Native Indicator	Cross-Platform Indicator	Hybrid Indicator
What are the performance requirements?	Real-time, animation-heavy, or hardware-intensive	Standard CRUD, moderate interaction	Simple forms, infrequent user interaction
What native APIs are required?	Deep integration: ARKit, HealthKit, CoreBluetooth, Face ID	Standard: push notifications, camera, GPS	Minimal: basic device info, web views
What is the team composition?	Dedicated iOS and Android engineers	Full-stack or JavaScript engineers	Web developers with no mobile experience
What are the UX requirements?	Platform-native conventions mandatory	Good UX acceptable; exact convention match not required	Functional over polished
What is the timeline to market?	Speed is less important than quality and maintainability	Speed matters; code sharing accelerates delivery	Fastest time to market with existing web team
What is the expected app lifespan?	5-10+ years; long-term maintainability critical	3-5 years; framework risk acceptable	1-3 years; experimental or internal tool

Platform Selection

Skill Check

Scenario A

Regional bank. Face ID, Apple Pay, camera deposit, WebSocket balances, 5-year support, 14 native engineers. What do you recommend and why?

Scenario B

Equipment rental company. Internal tool, 200 staff, Samsung tablets only, 3-month timeline, React web dev team. What do you recommend?

SOLID Design Principles

The Root Cause

- All five SOLID principles address the same underlying problem: coupling.
- Tight coupling = changes to one component force changes to another.
- SOLID = loose coupling + high cohesion.

All Five SOLID Principles at a Glance

Principle	Core Idea	Violation Signal	The Fix
S - Single Responsibility	One reason to change	Class has methods serving different 'actors'	Split into focused classes; each owns one concern
O - Open/Closed	Open for extension, closed for modification	Adding a new type requires editing existing code	Define a protocol/interface; add new types by adding new classes
L - Liskov Substitution	Subtypes must be substitutable for their base type	fatalError('not supported') in a protocol conformance	Prefer composition; don't extend what can't be substituted
I - Interface Segregation	No code should depend on methods it doesn't use	Implementing fatalError for unused protocol methods	Split fat protocols into smaller, focused ones
D - Dependency Inversion	Depend on abstractions, not concretions	Class creates its own URLSession/Database in its body	Inject dependencies via initializer; depend on protocols

Single Responsibility Principle

- A class should have only one reason to change
  - i.e. a class should be responsible to one, and only one, actor
- A TimesheetService class may do many things
  - e.g. fetch, create, update, and delete timesheets
  - All those things are the responsibility of one actor
- "If we change how emails are sent, do we also need to modify the TimesheetService"?
  - If the answer is yes, SRP is violated

S - Single Responsibility Violation

```
// VIOLATION: Five distinct reasons to change in one class
class TimesheetManager {
  func fetchTimesheets(employeeId: String) -> [Timesheet] {
    let url = "https://api.internal.com/timesheets/(id)"  // hardcoded!
    return parseJSON(URLSession.shared.dataTask(url: url))
  }
  func approveTimesheet(_ t: Timesheet) { ... }

  func exportToCSV(_ timesheets: [Timesheet]) -> String { ... }

  func sendEmail(to: String, body: String) { ... }

  func writeToAuditLog(_ msg: String) { ... }
}
```

S - Single Responsibility Fixed

```
class TimesheetRepository {
  func fetchTimesheets(employeeId: String) -> [Timesheet] { ... }
  func updateStatus(_ t: Timesheet, to status: TimesheetStatus) { ... }
}

class EmailNotificationService: NotificationService {
  func sendNotification(to recipient: String, body: String) { ... }
}

class AuditLogger {
  func log(_ message: String) { ... }
}
```

S - Single Responsibility Fixed

```
class TimesheetApprovalCoordinator {
  // Depends only on protocols - see DIP
  init(repository: ..., notificationService: ..., auditLogger: ...) { ... }

  func approveTimesheet(_ t: Timesheet) {
    repository.updateStatus(t, to: .approved)
    notificationService.sendNotification(to: t.employee.email, body: ...)
    auditLogger.log("Approved (t.id)")
  }
}
```

O - Open/Closed Principle

- Once a class is tested and deployed, you should make modifications to it by extending its behavior, not modifying its source code.
  - Modifications to existing code risk introducing regression bugs.
- Best mechanism to achieve OCP is abstraction
  - Define an interface (protocol in Swift) describing what some class does
  - Implement variations on the behavior in separate classes
  - Adding new behavior means adding a new implementation, not editing existing ones

O - Open/Closed Violation

```
class PaymentProcessor {
    func processPayment(amount: Double, type: String) {
        if type == "credit_card" {
            // credit card logic
        } else if type == "bank_transfer" {
            // bank transfer logic
        } else if type == "paypal" {
            // PayPal logic  <-- added this month
        }
        // Next month: else if type == "crypto" { ... }
    }
}
```

O - Open/Closed Fixed

```
protocol PaymentProvider {
    func processPayment(amount: Double) throws
}

class CreditCardPaymentProvider: PaymentProvider {
    func processPayment(amount: Double) throws { /* credit card logic */ }
}

class BankTransferPaymentProvider: PaymentProvider {
    func processPayment(amount: Double) throws { /* bank transfer logic */ }
}

class CryptoPaymentProvider: PaymentProvider {    // New type: new class only
    func processPayment(amount: Double) throws { /* crypto logic */ }
}
```

O - Open/Closed Fixed

```
protocol PaymentProvider {
    func processPayment(amount: Double) throws
}


class PaymentProcessor {

    private let provider: PaymentProvider

    init(provider: PaymentProvider) { self.provider = provider }

    func processPayment(amount: Double) throws {
        try provider.processPayment(amount: amount)
    }
}
```

L - Liskov Substitution Principle

- Objects of a subtype must be substitutable for objects of their parent type without altering the correctness of the program
- e.g. if you have a method that accepts a parameter of type Bird, you should be able to pass it an Ostrich (which extends Bird)
  - The method should still work correctly even though an Ostrich cannot fly
- This is the most subtle of the five SOLID principles
- Often appears in mobile codebases as a protocol (interface) method that throws a fatalError() in some conforming types
- Runtime type-checking is often a sign of a violation

L - Liskov Substitution Violation

```
class Rectangle {
    var width: Double
    var height: Double
    init(width: Double, height: Double) { self.width = width; self.height = height; }
    func area() -> Double { width * height }
}
class Square: Rectangle {
    override var width: Double {
        didSet { height = width }
    }
}
func testArea(_ shape: Rectangle) {
    shape.height = 4; shape.width = 5;
    assert(shape.area() == 20)          // FAILS for Square: area() returns 25
}
```

L - Liskov Substitution Fixed

```
protocol Shape {
    func area() -> Double
}

protocol Resizable {
    var width: Double { get set }
    var height: Double { get set }
}
```

L - Liskov Substitution Fixed

```
struct Rectangle: Shape, Resizable {
    var width: Double
    var height: Double
    func area() -> Double { width * height }
}

struct Square: Shape {
    var side: Double
    func area() -> Double { side * side }
}
```

I - Interface Segregation Principle

- No code should be forced to depend upon (implement) methods it does not use
- Prefer many small, specific interfaces over one large general one
  - Fat interfaces force implementing classes to provide no-op implementations of methods they do not need
  - A class that claims it can do something but throws an error when you try to do that thing violates the contract

I - Interface Segregation Violation

```
protocol DataSource {
    func fetchAll() -> [Record]
    func fetchById(_ id: UUID) -> Record?
    func insert(_ record: Record)
    func update(_ record: Record)
    func delete(_ id: UUID)
    func fetchPage(offset: Int, limit: Int) -> [Record]
    func count() -> Int
}
```

I - Interface Segregation Violation

```
class ReadOnlyRemoteDataSource: DataSource {
    func fetchAll() -> [Record] { /* network call */  return [] }
    func fetchById(_ id: UUID) -> Record { /* network call */  return nil }
    func insert(_ record: Record) { fatalError("Read-only!") }   // violation
    func update(_ record: Record) { fatalError("Read-only!") }  // violation
    func delete(_ id: UUID) { fatalError("Read-only!") } // violation
    func fetchPage(offset: Int, limit: Int) -> [Record] { return [] }
    func count() -> Int { return 0 }
}
```

I - Interface Segregation Fixed

```
protocol Readable {
    func fetchAll() -> [Record]
    func fetchById(_ id: UUID) -> Record?
}
protocol Writable {
    func insert(_ record: Record)
    func update(_ record: Record)
    func delete(_ id: UUID)
}
protocol Pageable {
    func fetchPage(offset: Int, limit: Int) -> [Record]
    func count() -> Int
}
```

I - Interface Segregation Fixed

```
class ReadOnlyRemoteDataSource: Readable, Pageable {
    func fetchAll() -> [Record] { /* network call */  return [] }
    func fetchById(_ id: UUID) -> Record { /* network call */  return nil }
    func fetchPage(offset: Int, limit: Int) -> [Record] { return [] }
    func count() -> Int { return 0 }


}
class LocalCacheDataSource: Readable, Writeable {
    func fetchAll() -> [Record] { return [] }
    func fetchById(_ id: UUID) -> Record { return nil }
    func insert(_ record: Record) { . . . }
    func update(_ record: Record) { . . . }
    func delete(_ id: UUID) { . . . }
}
```

D - Dependency Inversion Principle

- High level modules should not depend on low level modules
  - Both should depend upon abstractions
- Abstractions should not depend upon details (implementations)
  - Details should depend upon abstractions
- This is the foundation of testability
  - When a class directly instantiates its dependencies, they cannot be replaced in tests
  - This makes it impossible to create true unit tests
- When a class depends upon an abstraction instead, the implementation can be passed in from outside the class
  - The application can inject a real implementation at run time
  - Unit tests can inject a mock implementation for testing purposes

D - Dependency Inversion Violation

```
class LoginViewModel {
    private let networkService = NetworkService()         // concrete, hardcoded
    private let analyticsService = FirebaseAnalytics()    // concrete, hardcoded

    func login(email: String, password: String) async {
        let result = await networkService.post("/auth/login",
            body: ["email": email, "password": password])
        analyticsService.track(event: "loggin_attempted")
        . . .
    }
}
```

D - Dependency Inversion Fixed

```
protocol NetworkServiceProtocol {
    func post(_path: String, body: [String: Any]) async -> Result<Data, Error>
}
protocol AnalyticsServiceProtocol {
    func track(event: String)
}
```

D - Dependency Inversion Fixed

```
class loginViewModel {
  private let networkSerivce: NetworkServiceProtocol
  private let analyticsService: AnalyticsServiceProtocol

  init(networkService: NetworkServiceProtocol, analyticsService: AnalyticsServiceProtocol) {
    self.networkService = networkService
    self.analyticsService = analyticsService
  }

  func login(email: String, password: String) async {
    . . .
  }
}
```

D - Dependency Inversion in Tests

```
class MockNetworkService: NetworkServiceProtocol {
    var responseToReturn: Result<Data, Error> = .success(Data())
    func post(_path: String, body: [String: Any]) async -> Result<Data, Error> {
        return responseToReturn
    }
}

let viewModel = LoginViewModel(
    networkService: MockNetworkService(),
    analyticsService: MockAnalyticsService()
}
```

SOLID Principles Applied

- They are not independent
  - In well-designed code, they reinforce each other
- A codebase that honors all five principles can be changed fearlessly
  - You can add features, modify a service, or swap a dependency without touching unrelated code
  - You do not need to keep a spreadsheet list of "things that might break"

SOLID Principles Audit

Skill Check

The Code

TimesheetManager class in the GitHub repository.

Five Questions

Q1: All violations + harm. Q2: Why is fetchTimesheets untestable? Q3: Slack migration - how many files change? Q4: Design a refactored version. Q5: Write one unit test.

Clean Code and Technical Debt

Meaningful Names - The Rules

Category	Rule	Bad	Good
Variables	Reveal intent - what does this represent?	let d = 86400	let secondsInDay = 86400
Booleans	Read as a true/false assertion	let flag = true	let isAuthenticated = true
Functions	Verb-noun pair - describe what it does	func process()	func fetchApprovedTimesheets()
Magic Numbers	Named constants, always	if attempts > 3 {	if attempts > maxLoginAttempts {
Parameters	Same rules as variables - no single letters	func f(a: Int, b: String)	func calc(hours: Int, name: String)

Functions

- Robert Martin's rules for functions:
- Functions should be small (5-15 lines)
  - If you must scroll, it is too long
- Functions should do one thing
  - If you can extract a meaningful sub function from it, it was doing more than one thing
- One level of abstraction per function
  - Don't mix high-level intent with low-level implementation in the same function
- Functions should have no side-effects
  - A function named getUser should not also log to a file or increment a counter
- Prefer fewer parameters. Zero is ideal. One is good. Two is acceptable.
  - Three or more requires strong justification
  - More than three is almost always wrong. Group the values into an object.

Function Violations

```
// VIOLATIONS: too many parameters, vague name, multiple side-effects

func manage(id: String, mode: Int, flag: Bool, count: Int, data: [String: Any]) -> Bool {
    if mode == 1 {
        log("Entering manage mode 1")
        // . . . 40 lines of code
        sendEmail(to: data["email"] as! String, body: "Done")
        return true
    }
    . . .
}
```

Function Fixed

```
// CORRECTED: small, focused, descriptive functions with clear names

func approveTimesheet(_ timesheet: Timesheet) {
    repository.updateStatus(timesheet, to: .approved)
    notificationService.sendApprovalNotification(for: timesheet)
}

func sendApprovalNotification(for timesheet: Timesheet) {
    let message = buildApprovalMessage(for: timesheet)
    emailService.send(to: timesheet.employee.email, body: message)
}
```

Comments

- The goal is not to avoid all comments - only those that exist because the code is unclear

Comment Type	Rule	Example
Legal comments	Acceptable. Required for copyright, license, authorship.	// Copyright (c) 2026 PNC Bank
Intent comments (why)	Acceptable. Explains why a decision was made, not what the code does.	// We use 45s timeout here because the Apigee P95 latency under load is 38s (see MOB-412)
Warning comments	Acceptable. Warns future engineers of consequences.	// WARNING: Do not call this method on the main thread - it blocks on network I/O
TODO comments	Use sparingly. Track TODOs in the backlog, not the code.	// TODO: Remove this workaround once iOS 16 support is dropped (MOB-892)
Explanatory comments (what)	Avoid. Rewrite the code instead.	// Increment counter by 1 → i += 1 (the code says this)
Noise comments	Never. They add noise and go stale.	// Default constructor → constructor with no parameters
Commented-out code	Delete it. Version control is your history.	// let oldService = LegacyTimesheetService()

Error Handling

- Professional code makes error handling explicit
  - Unhandled errors are production incidents waiting to be discovered
- Never swallow exceptions.
  - An empty catch block is a lie - it tells the runtime (and future readers) that nothing bad happened, when in fact something bad happened and you ignored it.
- Provide context in error messages.
  - 'Error' is not a useful error message.
  - 'Failed to fetch timesheets for employee E12345: connection timed out after 30s' is.
- Distinguish between recoverable and unrecoverable errors.
  - A network timeout is recoverable (retry).
  - A corrupted data store is not (show error, log, alert on-call).
- Error handling should not obscure the happy path.
  - If your function is 60% error handling code, restructure it.

Error Handling Violation

```
// VIOLATION: swallowing an error silently

func fetchTimesheets() -> [Timesheet] {
    do {
        return try repository.fetchAll()
    } catch {
        return []          // the caller has no idea something went wrong
    }
}
```

Error Handling Fixed

```
// CORRECTED: surface the error explicitly

func fetchTimesheets() -> Result<[Timesheet], TimesheetError> {
    do {
        let timesheets = try repository.fetchAll()
        return .success(timesheets)
    } catch let error as RepositoryError {
        logger.error("Failed to fetch timesheets: \(error.localizedDescription)")
        return .failure(.repositoryUnavailable(underlying: error))
    } catch {
        return .failure(.unknown(underlying: error))
    }
}
```

Code Formatting and Consistency

- Code formatting is a team concern, not an individual preference
  - A codebase that uses different formatting conventions in different files - because each engineer has their own preferences - imposes cognitive overhead on every reader and produces noisy diffs that obscure real changes
- The solution is to automate formatting enforcement:
  - iOS: SwiftLint for linting (error and warning rules); swift-format for automated formatting
  - Android: ktlint for Kotlin linting and formatting; Android Studio's built-in formatter
- Run formatters and linters in the CI pipeline - a build that fails lint is not mergeable
- Never debate formatting in code review
  - If the linter accepts it, it's acceptable. If it doesn't, fix it.

Technical Debt - Fowler's Quadrant

- Like financial debt, technical debt is a useful tool when used deliberately and paid back promptly
  - It is also a dangerous liability when it accumulates uncontrolled

	Prudent	Reckless
Deliberate	"We'll ship now and refactor post-launch" - acceptable if tracked and paid	"No time for design" - shortcuts with no plan to pay back
Inadvertent	"Now we see how we should have done it" - normal learning; address promptly	"What is layered architecture?" - absent knowledge that should have been present

Common Sources of Technical Debt in Mobile Engineering

Source	Description	Compounding Effect
Missing tests	Features shipped without unit tests. Refactoring the code later is risky because there is no safety net.	Each refactor without tests risks breaking existing behavior silently. Over time, engineers stop refactoring out of fear. Code solidifies in bad states.
Copy-paste programming	Logic duplicated because extraction was considered too slow. The same bug now lives in ten places.	Every bug fix must be applied to every copy. Every feature change must be propagated. The team loses track of which copy is authoritative.
God classes	A single class (often named Manager, Handler, or Controller) that knows everything and does everything.	Every feature request touches the god class. Merge conflicts on this file are constant. Testing is nearly impossible because the class has hundreds of dependencies.
Outdated dependencies	Third-party libraries or platform SDKs that have not been updated. Security vulnerabilities accumulate; API deprecations accumulate.	When a critical security fix requires a major version upgrade, the accumulated API changes make the upgrade a weeks-long project instead of a day's work.
No API versioning strategy	Mobile apps talk directly to v1 of an API with no migration plan. When the backend team changes the API, every app in the field breaks.	Each backend change requires a coordinated forced-update campaign to ensure users are on the new app version - which many users resist.
Inadequate error handling	Errors swallowed silently or handled with generic fallbacks. Production issues are invisible until users complain.	Error rates climb without triggering alerts. The team learns about bugs from support tickets, not monitoring dashboards. Root cause analysis becomes guesswork.

Measuring Technical Debt

- You cannot manage what you cannot measure.
- Code complexity metrics: Cyclomatic complexity measures the number of independent paths through a function. A function with cyclomatic complexity above 10 is difficult to test and understand. Static analysis tools (SonarQube, SwiftLint rules) can report this automatically.
- Test coverage: The percentage of code lines executed by at least one test. Coverage below 60% on a codebase in active development is a red flag. Coverage above 80% on the core business logic layer is a reasonable target.
- Code duplication percentage: Tools like PMD CPD (Copy-Paste Detector) identify duplicated code blocks. High duplication percentages indicate violation of the DRY (Don't Repeat Yourself) principle.
- Dependency staleness: The average age of third-party dependencies relative to their current release. Anything more than two major versions behind warrants investigation.

Managing and Paying Down Technical Debt

Strategy	Description	When to Apply
Boy Scout Rule	Every time you touch a file, leave it slightly cleaner than you found it. Rename an unclear variable. Extract a long method. Add a missing test for a function you read.	Every pull request, every day. Not a separate activity - a continuous habit.
Debt backlog	Maintain a section of the product backlog dedicated to debt reduction work. Each item is described, estimated, and prioritized alongside features. It is visible to the Product Owner.	Whenever a debt item is significant enough to warrant dedicated sprint capacity.
Dedicated refactoring sprints	Occasionally run a sprint (or part of a sprint) focused entirely on technical debt reduction. No new features. Only cleanup, testing, and architecture improvements.	When debt has accumulated to the point of materially slowing feature delivery - typically quarterly or semi-annually.
Incremental strangler pattern	For large rewrites, incrementally replace legacy code by routing traffic through a new implementation while the old one is still running. New and old coexist until the migration is complete.	When an entire component must be replaced but cannot be taken offline for a full rewrite.
Hard DoD requirements	Enforce code coverage thresholds, linting rules, and complexity limits in the CI pipeline. Builds that violate these requirements fail. Debt cannot be merged.	Continuously, from the start of a project. Retrofitting these requirements on an existing codebase is painful - preventing debt is easier than curing it.

Knowledge Check

Knowledge Check · Question 1

Describe the difference between functional and non-functional requirements. Give two examples of each for a mobile timesheet application.

Knowledge Check · Question 2

A product manager says: 'We are an Agile team, so we don't need design documentation.' How do you respond with a precise, evidence-based correction?

Knowledge Check · Question 3

Explain the Dependency Inversion Principle. Why does it directly enable unit testing, and what specific technique implements it?

Knowledge Check · Question 4

A team's Sprint Review reveals 4 of 7 committed stories are not Done. The Scrum Master says: 'We'll carry them over and count them in next Sprint's velocity.' What is wrong with this approach?

Knowledge Check · Question 5

What is the Test Pyramid? Why does an 'inverted pyramid' (many UI tests, few unit tests) indicate a maintenance problem?

Source Control and Team Development

Module 2

Version Control Fundamentals and Git Internals

Why Version Control? Five Problems It Solves

- **Lost work** - a colleague overwrites your changes with an older file version
- **Mystery bugs** - a defect appears but no one knows what changed or when
- **Integration horror** - two engineers merge two weeks of parallel work manually
- **No history** - the only copy of working code is the one currently in production
- **No blame** (the useful kind) - you can't trace who changed what and why

What is a Version Control System (VCS)?

- Software tool that tracks the history of a group of files.
- It keeps track of every change made to any of the files in a special database.
- This database is called a repository.
- VCSs support creating different versions of the group of files.
  - Each version is a collection of snapshots of the state of each file.
- They allow switching between the versions.
- They support merging changes from different versions in a wide variety of ways.

Why Use a VCS?

- They facilitate a smooth flow of changes to the files.
  - They help manage multiple developers working on the files at the same time.
- If mistakes are made in the files, the state of the collection can be rolled back.
- They make it easy to maintain multiple versions of the set of files.
  - Changes can be merged into any or all of the versions.
- They allow you to see the differences between different versions of files.
  - This includes the complete history of changes to a given file.
- They support experimenting with new features without affecting production code.
- They let you view how many changes are made, to which files, when and by whom.
- They allow you to pause development of a feature to fix a bug.
  - And then merge the bug fix into production code as well as the feature branch.

Centralized vs. Distributed VCS

- Centralized VCS have one authoritative repository.
  - Developers can check out (lock) versions of files for modification.
  - When finished making changes, files are checked back in.
  - Locked files prevent conflicting changes by multiple developers.
  - Have a single point of failure (the centralized repository).
- Distributed VCS treat all repositories as peers.
  - Each developer has a local clone of the entire repository.
  - Files are not checked out or locked.
  - Local copies can be synchronized to any remote repository.
  - One central clone is typically used as the "authoritative" copy.
  - Conflicting changes must be merged (frequently a manual process).
  - Conflicts are minimized through the use of branches and rebasing.

Git

- Is the most popular implementation of a distributed VCS.
- Was developed in 2005 from Linux kernel development by Linus Torvalds and the Linux community.
- Is used by many popular open-source projects.
- Is the VCS utilized by GitHub and BitBucket.
  - They are online hosts for Git repositories (repos).
  - Other platforms such as Microsoft's DevOps support the Git VCS.

More About Git

- Git stores snapshots of files, not a list of changes.
  - Most other VCS store changesets.
- Everything in Git is checksummed using a SHA-1 hash.
  - Data is stored in Git not only with its filename, but also with the 40-character hash.
  - You cannot lose or corrupt anything without Git knowing about it.
- Git generally only adds data.
  - It is hard to get Git to do anything that is undo-able or to remove data.
  - Once you commit something into Git, it is difficult to lose.
- The .git directory is where your project's metadata and object database is stored.
  - This is what is copied when the project is cloned or created when a new repo is created.

Git Repositories (repos)

- Store the history of a collection of files.
- Each application, module or library would typically be its own repository.
- Copying an entire repo is called cloning.
  - E.g. you would clone the server repo to get a local copy for development.
- Some repos are Bare repositories.
  - They contain only Git objects (no .py or .cs or .java files).
  - They cannot be directly modified.
  - They typically serve as a central "authoritative" copy for developers to synchronize.
- Other repos are Non-Bare repositories.
  - They contain all the source code files as well as the Git objects.
  - They can be directly modified.
  - They are used locally for developers' modifications.

The Four Git Object Types

Type	Contains	Identified By
Blob	The raw content of a file. No filename, no metadata - just bytes.	SHA-1 hash of its content. Identical files share one blob.
Tree	A directory listing: (mode, name, hash) entries pointing to blobs or other trees.	SHA-1 hash of its content. Represents a directory at one point in time.
Commit	A pointer to a root tree + author, committer, timestamp, parent hash(es), message.	SHA-1 hash. The parent hash is why history is immutable.
Tag	An annotated pointer to any object (usually a commit) with tagger, date, message.	SHA-1 hash.

Why This Matters

- Because each commit's hash includes its parent's hash, you cannot change a commit without changing its hash - and every subsequent commit.
- Rewriting history doesn't modify existing commits. It creates new ones and removes the reference to the old ones.

The Three Trees - Working Directory, Index, Repository

Tree	What It Is	Modified By
Working Directory	The actual files on your disk right now - what you see in your editor.	You - by editing files
Index (Staging Area)	A proposed snapshot of the next commit. git add promotes changes here.	git add, git rm, git restore --staged
Repository (HEAD)	The last committed snapshot - the current commit HEAD points to.	git commit - promotes Index to Repository

Branches

- Branches are versions of the complete collection of files in a repo.
- Each branch is a fork in the history of the project.
- New commits are recorded in the history of the current branch.
- Developers can check out different branches at different times.
- Branches allow developers to work on different changes independently.
- Local branches can have a counterpart in a remote repo.
  - That would be called a remote-tracking branch.
- Git can combine changes from one branch into another.
  - This action is commonly called merging.
  - There is a variation called rebasing.

More on Branches

- They are an abstraction for the edit/stage/commit process.
- Branches are where all work gets done.
- They are used to isolate work from other work.
- Compared to some other VCS, branches are cheap to create.
  - Both in time and in disk space.
  - It is generally recommended to branch early and often.
- They function to keep the main branch clean.
  - You should never have unstable (untested/buggy) code on the main branch.
- Each branch is a named pointer to a commit.
  - A commit is a snapshot of the state of all the files in the repo.
  - Git maintains a pointer called HEAD, which is a symbolic reference to the currently checked-out branch.

Installing Git

- Git can be downloaded from https://git-scm.com/downloads.
  - You will find downloads for each O/S there.
  - On Linux you can also install Git using the package manager for your distribution.
    - E.g. dnf or apt
  - On MacOS 10.9+ attempting to invoke git from the Xcode command line will prompt for installing it, if it is not present.
- An alternate option exists for Windows and MacOS.
  - You can install GitHub Desktop from https://desktop.github.com/
  - This includes both a CLI called Git Bash and a GUI client interface.
  - On Windows, this is often the best choice for most developers.

Git GUI Clients

Name	Price	Windows	Mac	Linux
SmartGit	$0 - $239	X	X	X
GitKraken	$0 - $49/yr	X	X	X
Git Extensions	FREE	X	X	X
SourceTree	FREE	X	X	
Fork	FREE	X	X	
TortoiseGit	FREE	X		
Git For Windows	FREE	X		

Getting Started

Skill Check

- If necessary, download and install Git for your O/S. You can use the downloads from:
  - https://git-scm.com/downloads
- For Windows, you can use:
  - https://desktop.github.com
- Optionally, download and install one of the Git GUI programs presented in the lesson.

Core Git Operations - Daily Workflow

Configuration

- Git has three levels of configuration:
- System-wide settings apply to how Git works for every user on the computer.
  - Stored in the /etc/gitconfig file.
  - Use the --system parameter to work with these settings.
- Global settings apply to all repos for the current user.
  - Stored in either the ~/.gitconfig or the ~/.config/git/config file.
  - Use the --global parameter to work with these settings.
- Repository settings apply to a particular repo.
  - Stored in .git/config in the particular repo.
  - Use the --local parameter to work with these settings (or omit any parameter, as this is the default.)
- When resolving conflicts, the more local settings override the more global settings.

Changing Configuration Settings

- Use the git config command in the bash console to change settings.
- Commonly-used commands include:
  - --get - retrieves current values.
  - --unset - removes settings.
- Commonly-changed settings include:
  - user.name
  - user.email
  - core.editor
  - commit.template
  - help.autocorrect

Code Bite

```
git config --global user.name "John Doe"
git config --global user.email
  "john.doe@example.com"
```

The Three Trees of Git

- The Working Directory
  - The state of the local filesystem.
  - Represents changes made to files and directories.
- The Staging Index
  - Subset of changes in the working directory.
  - Those that have been promoted with git add
  - Marked to be included with the next commit.
  - Can be viewed using git ls-files -s
- The Commit History
  - Collection of permanent snapshots of the states of files.
  - Can be viewed using git log

Creating a Repository

- The init command creates a new repository in an empty directory.
- It can also turn an existing project into a Git repository.
- It adds a .git directory with a config file and a Git database.
  - You can create bare repos.
  - You can copy in files from another directory to seed the new repo.
- The new repo has no remote connection yet.

Code Bite

```
git init


git init --bare


git init <directory> --template =
<template_directory>
```

.gitignore File

- It is a text file located in the top-level of the repo.
- Contains descriptions of files and directories Git should ignore.
  - E.g. should exclude the "bin" directory that contains compiled output.
- Use a hashtag (#) to create comments.
- Brackets [] surround choices.
- Asterisk (*) indicates any number of characters.

Code Bite

```
# Build results
[Dd]ebug/
[Rr]elease/
[Bb]in/
[Oo]bj/
[Ll]og/

# User-specific files
*.user
*.userosscache
```

Staging and Committing

- Persisting changes to the local repo is a two-stage process in Git.
- Changes to files get staged.
  - Selected changes to files get added to the index (staging area).
  - Anything from all changes to a given set of files down to one line of code changed in one file.
- Staged changes are then committed.
  - Changes are added permanently to the repo in a persistent snapshot.
  - Commits are history points to which you can revert at any point in time.
- Files with staged changes can continue to be edited.
  - Only the staged changes will get committed.
  - Unstaged changes will remain in the working directory.

Staging Changes

- The add command moved one or more files from the working directory to the staging index.
- Changes can be staged multiple times before committing.
- The status command lists out files with staged/unstaged changes.

Code Bite

```
git status

git add .

git add readme.md another_file.cs
```

Adding Commits

- At any time, staged changes can be committed to the repo.
- A commit is a "saved" version of the set of files.
  - Identified by a unique ID (SHA-1 hash).
  - Can be reverted to at any time.
- Common options include:
  - -m - sets a commit message.
  - --amend - updates the previous commit instead of creating a new one.

Code Bite

```
git commit -m "Adds ReadMe and a class"

git commit --amend
```

Guidelines for Commit Messages

- A commit message should have a header and a body.
  - The header should be less than 50 characters, and the body should wrap at 72 characters.
  - The body should be separated from the header by an empty line.
  - This will display well on the command line and in graphical tools.
- The body should describe the reason why the change was made.
- The commit message should be in present tense.
  - E.g. "Adds better error handling" instead of "Added better error handling".
- The last paragraph can also contain metadata as key-value pairs.
  - This data is also referred to as the commit message footer.
- This metadata is often used for storing data for later searches.
  - E.g. keys of 'Change-Id', 'Signed-off-by' or 'Bug'.

Commit Message Examples

- Example of what to avoid:
  - 21a8456 update
  - 29f4219 update
  - 016c696 update
  - 29bc541 update
  - 740a130 initial commit

- Better example:
  - 7455823 Adds search and filter to the model editor tree
  - 9a84a8a Replaces missing DynamicMenuContribution in child selector
  - 952e014 Fixes spelling error in Toolbar/Add child
  - 71eeea9 Adds option to import model elements from legacy RCP
  - 123672c New Application wizard is missing dependencies
  - 97cdb9a Creates an id for handlers

Viewing Commit History

- The log command displays the history of commits for the current branch.
- This can result in a lot of data, so Git displays one page at a time.
  - The spacebar scrolls down a page at a time.
  - Press q to quit the log output.
  - Don't use ctrl-c to quit - it often mutes the output of typed commands.
  - If this happens, use the reset command.

Code Bite

```
git log
```

Log Options

- Commonly-used options include:
  - --oneline
  - -3 - most recent 3 commits (or any number)
  - --author <name>
  - --committer <name>
  - --before <date>
  - --after <date>

Code Bite

```
git log --oneline

git log --oneline -5

git log --oneline --after 2020-01-01

git log --oneline --after 3.months.ago
```

Other log Options

- The --stat option displays a summary of the changes included in each commit.
- The -p option displays details of changes included in each commit.
  - What was added.
  - What was removed.
  - Changes show up as a removal and addition.
  - Who made the change and when.
  - Who made the commit and when.

Code Bite

```
git log -1 --stat

git log -1 -p
```

Viewing Prior Versions of Files

- The show command can be used to view details of any git object.
- It can be used to refer back to versions of files from any prior commit.
  - Specify the commit hash (or a symbolic link such as HEAD) followed by a colon and the path to the file.
- They can be compared with the current version or the previous commit's version.

Code Bite

```
git show ff04b9e:./wwwroot/css/styles.css

git show HEAD:./wwwroot/js/site.js
```

Stashing Changes

- When changing branches, uncommitted changes will be lost.
  - If you are not ready to commit the changes, but do not want to lose them, they should be stashed.
- The Git stash is a stack of file snapshots that is used to store changes you do not wish to commit yet.
- Stashed changes are stored ONLY in your local repo.
  - They do NOT get copied to a remote repo when pushing.
- Stashing will preserve changes to any file currently tracked by Git (staged or unstaged).
- Stashed changes can later be applied to your working copy.
  - Popping the stash will apply the changes and remove them from the stash.
  - Applying the stash will apply the changes but leave them in the stash to be applied elsewhere.

The stash push Command

- The stash push command saves current, uncommitted changes for later use.
  - Includes both staged and unstaged changes.
  - Reverts them out of your working copy.
  - It is the default stash command and can be omitted.
- Will NOT preserve changes to new files that are not yet staged.
  - The -u option (or --include-untracked) will include untracked files.
- Will NOT preserve changes to ignored files.
  - The --all option will include ignored files and untracked files.

Code Bite

```
git stash push

git stash -u

git stash -all
```

Managing Multiple Stashes

- Each stash you create gets added to the top of a stack.
- The list command displays the stack of stashes.
- Descriptions should be added, with the -m option, to better manage them.
  - Names like "stash@{0}", "stash@{1}" are not very helpful.
- The show command will display a summary of the stash.
  - The -p switch will display the full diff for the stash.

Code Bite

```
git stash -m started-feature


git stash list


git stash show -p


git stash -m did-some-silly-things
```

Other stash Commands

- The pop command removes a stash from the list and applies it to the working directory.
- The apply command applies a stash to the working directory without removing it from the list.
- The branch command will create a new branch by applying the stash to the current branch.
- The drop command will get rid of a stash without applying it.

Code Bite

```
git stash pop

git stash apply

git stash branch new-branch started-feature

git stash drop did-some-silly-things
```

Stopping Tracking a File

- Changes to .gitignore will only affect whether Git tracks each new file as they are added to the repo.
- Files that are already being tracked that match the newly added .gitignore rule will still be tracked by the repo going forward.
- The rm command tells Git to stop tracking a file that it is currently tracking.

The rm Command

- Tells Git to stop tracking a file, or a collection of files.
- Removes the file from the working tree and the staging index.
  - --cached removes only from the staging index, leaving the file in the working tree.
- The file must be identical to the tip of the branch.
  - If there are edits to the file in the staging index, the removal will fail.
  - -f will override this check.
- These changes are not permanent until the next commit occurs.
  - git reset HEAD would undo them.

Code Bite

```
git rm documentation/notes.txt

git rm --cached www/scripts/module.js
```

Clean Command

- Removes untracked files from the current branch.
- Can be executed as a dry run.
  - Use the -n switch.
  - Tells you what it would delete without actually deleting it.
- Refuses to work unless one of its switches is used also.
  - Safety net so you do not accidentally use it.
  - The -f switch forces deletions to go ahead.
  - The -d switch removes untracked directories.
  - The -x switch removes ignored files.

Code Bite

```
git clean

git clean -n

git clean -f
git clean -fn

git clean -d
git clean -dn
```

Creating a new Branch

- The branch command lists all branches.
- Supplying a branch name will create a new branch.
  - The new branch uses the current branch as its starting point.
  - This does not check out the new branch.
- The -d option will delete a branch.
  - Deletion will fail if the branch contains uncommitted changes.
  - The -D option will force deletion in that case.
- The -m option will rename the current branch.

Code Bite

```
git branch

git branch my-new-branch

git branch -d branch-to-delete

git branch -D branch-to-force-delete

git branch -m new-name-for-branch
```

Changing Branches

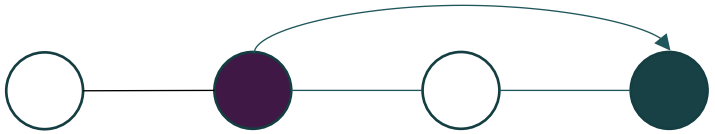
- The checkout command is used to set the current, active branch.
  - Changes, staging and commits will apply to the current branch.
- The -b option will simultaneously create and checkout a new branch.
  - The new branch will be based upon the current branch unless specified otherwise.
- You can checkout a specific commit by specifying the commit id instead of branch name.
  - This will put you in a detached HEAD state.
  - HEAD points to a commit, not a branch.
  - Code changes will not get saved to any branch - beware!

Code Bite

```
git checkout my-feature-branch


git checkout -b my-new-branch
git checkout -b new-branch base-branch


git checkout a1e8fb5
```

Revert Command

- The revert command "undoes" a previous commit (or multiple commits) by applying offsetting changes.
- It figures out how to undo changes and applies a new commit that offsets changes.
  - This preserves the history that the changes were once there.
- The --no-commit option applies inverse changes to the working tree and staging only (does not commit those changes).

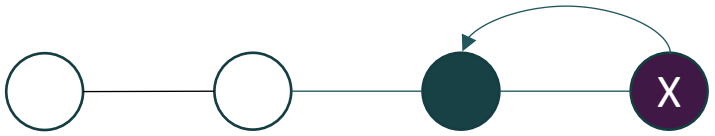


Code Bite

```
git revert

git revert 369b179


git revert --no-commit
```

Reset Command

- Revert simply offsets one or more commits.
  - Does not revert to a previous state.
  - Can offset any commit from any point in the branch history.
- Reset will revert the project back to a previous state.
  - Removes all subsequent commits.
- Is a destructive edit
  - "Alters" history by removing commits.
  - Can only work backwards from current state.

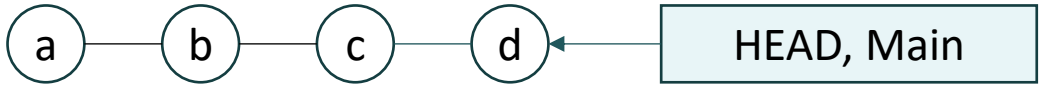


Code Bite

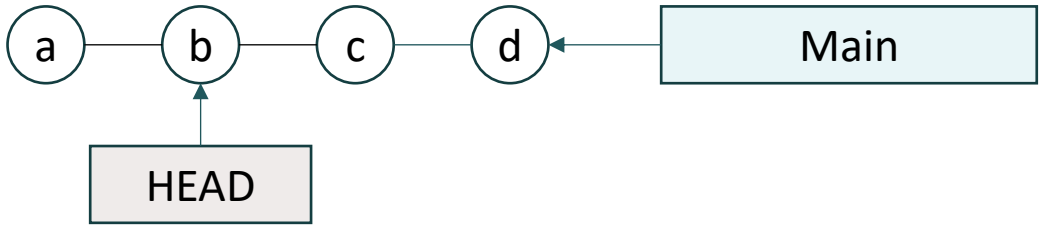
```
git reset
git reset --mixed HEAD

git reset --hard 369b179

git reset --mixed 369b179

git reset --soft 369b179
```

How Reset Works

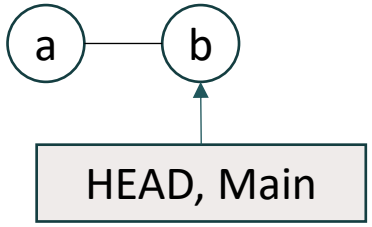
- Git maintains both a HEAD ref pointer and a current branch ref pointer.



- Using checkout with a commit ID only moves the HEAD ref.
  - You are in a detached HEAD state at this point.



- Using reset with a commit ID moves both HEAD and current branch ref pointer.
  - The state(s) of the three trees are also modified.



Reset Options

- --hard
  - Most frequently used (and most DANGEROUS) option.
  - Both Staging Index and Working Directory are reset along with the Commit History.
  - All pending work is lost.
- --mixed
  - Default option (used if no parameters are passed).
  - Commit History is reset, changes in Staging Index are moved to Working Directory.
  - Pending work is still available in the Working Directory.
- --soft
  - Commit History is reset, and nothing else.
  - Staging Index and Working Directory are left untouched.
- NEVER reset to a commit when any snapshots after it have been published.
  - Once something is published to a public repo, you must assume other developers are dependent upon it.

Basic Git Commands

Skill Check

- Find and follow the instructions markdown document in the GitHub repository for the Basic Git Commands repo.

Git - Working with Remotes

Listing Existing Remotes

- Your repos can have more than one remote repo associated.
  - You can push or pull changes to any of the configured remotes.
- The remote command lists out all remotes for the current repo.
- The -v option will include the URLs that Git uses for communicating with each remote.
- Details of a given remote can be listed using the remote show command.
  - It has one parameter - the name of the remote.

Code Bite

```
git remote

git remote -v

git remote show origin
```

Adding a Remote

- The remote add command registers a new remote repo.
- It takes two parameters:
  - What you would like to name the new remote.
  - The URL for communicating with the new remote.

Code Bite

```
git remote add df
 https://github.com/some-user/some-project

git remote add
 team https://10.23.55.108/our-project
```

Removing Remotes

- The remote rename command will rename a registered remote repo.
  - This changes nothing on the remote repo, it just changes the local name used to refer to it.
  - It has two parameters - the current name of the remote and the new name.
- The remote remove command will remove a registered remote repo.
  - Again, this does nothing to the remote repo; it just removes the local registration for it.
  - It takes one parameter - the name of the remote repo.

Code Bite

```
git remote rename df drew

git remote remove drew
```

Cloning a Repository

- One way of getting started with Git.
- The clone command copies an entire Git repo to another location.
  - Automatically creates a remote connection called "origin" that points to the original source location.
- Long-lived projects can develop a large history of commits.
- Shallow clones will only copy over a truncated history of commits.
  - The depth parameter specifies the number of recent commits to include.
  - Or the shallow-since parameter specifies the earliest commits to include.

Code Bite

```
git clone
 https://github.com/some-user/some-repo

git clone -depth=1
 https://github.com/some-user/some-repo
```

Forking a Repository

- Similar to a clone, a fork is a hosted copy of a hosted repository.
- Used to allow contributions by anyone to managed projects in a curated way.
  - Admins of the original repo are in control of if/when pull requests are merged into codebase.

Steps

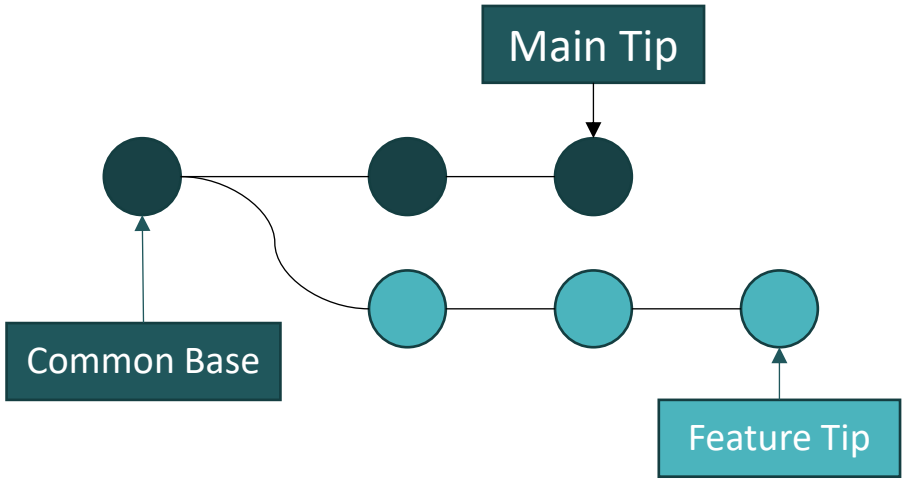
1. Create a fork of the desired repo.
2. Clone the fork locally.
3. Develop locally.
4. Push changes to the fork.
5. When desired, issue a Pull Request to merge the fork into the original repo.

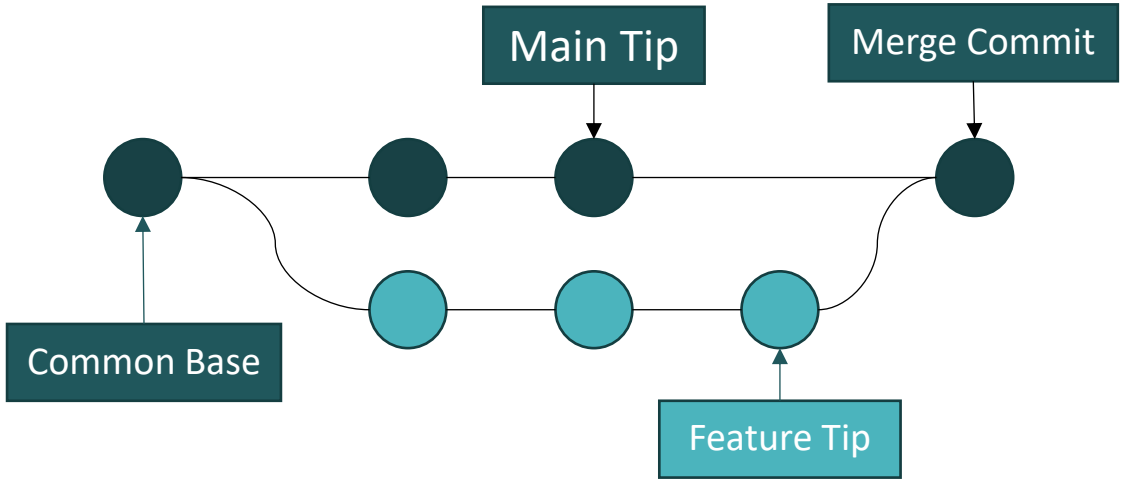
Push/Pull/Fetch

- All are Git commands to move changes from one location to another.
  - Differ in where the changes come from and where they are going.
- Distributed VCS like Git allow for changes to happen in any copy of the repo.
- These commands help to bring the disparate copies into convergence.
- They operate on the currently checked out branch of the repo.
  - If more than one remote repo is tracked, the desired remote needs to be specified.
  - If some other branch is desired, it can be specified instead of the current branch.
- Understanding what these commands to requires understanding the Merge command.

Merge

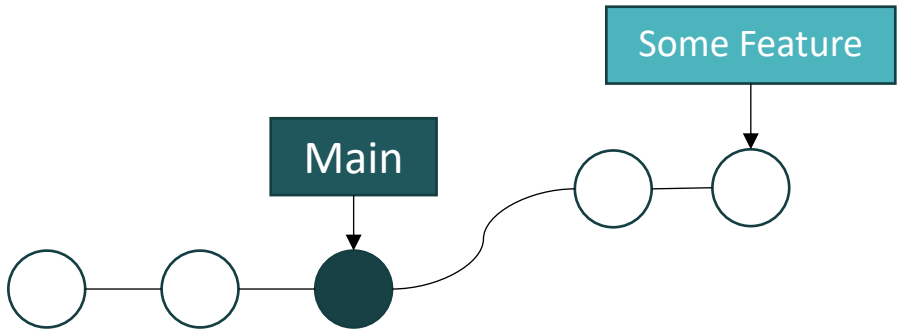
- Combines multiple sequences of commits into one unified history
- They are used to combine branches together.
  - Combines all the commits from the divergence point on one branch into a new commit on the other branch.
  - Merge commits are unique in that they have two parent commits.
- They can fail if there is a piece of data that is changed in both histories.
  - This would require human intervention to resolve the merge conflicts.

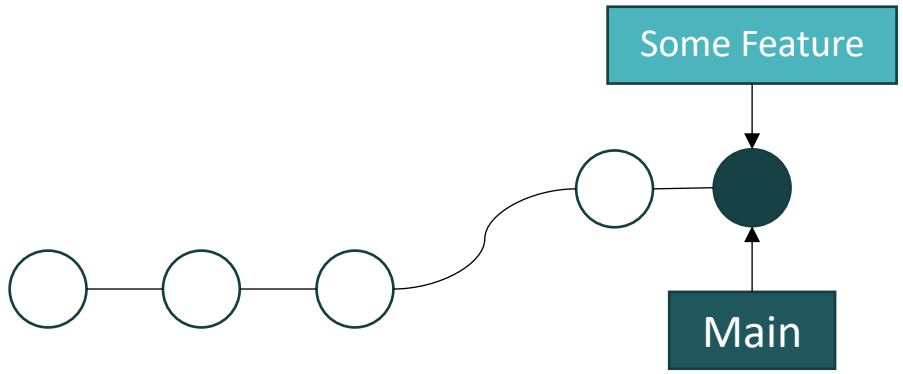




Fast-Forward Merge

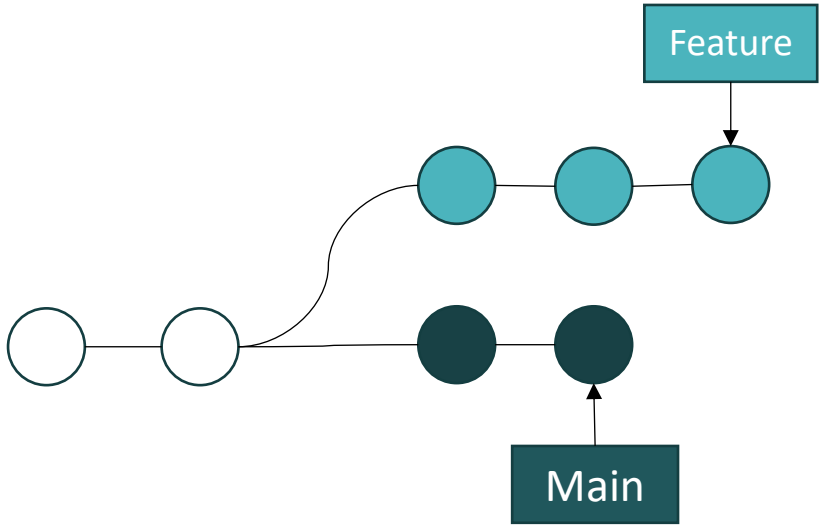
- Can occur when not combining multiple sequences of commits.
  - i.e. commits have only happened on one branch.
  - There is a linear path from the target branch tip to the current branch tip.
- Git just has to move the target branch tip to the current branch tip.
- This is the type of merge used by the push, pull and sync commands.

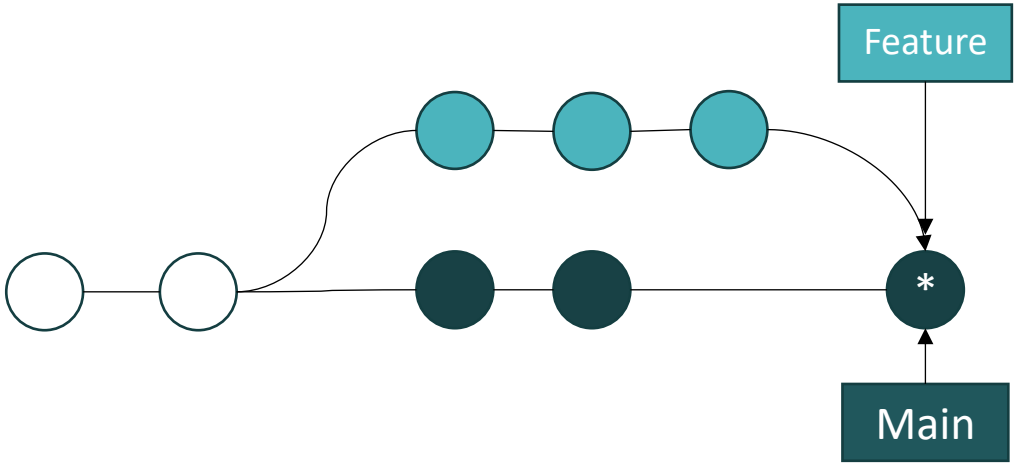




Merge Commit

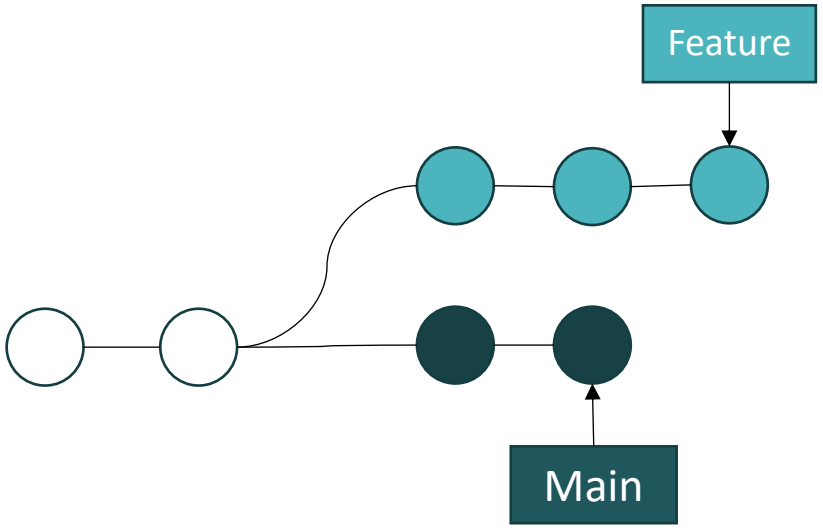
- Merge commits are a non-destructive operation.
  - All commits (changes) are preserved.
  - Preserves all history of when main branch changes were incorporated into the feature branch.
  - If main branch is very active, then the feature branch's history can become very convoluted.

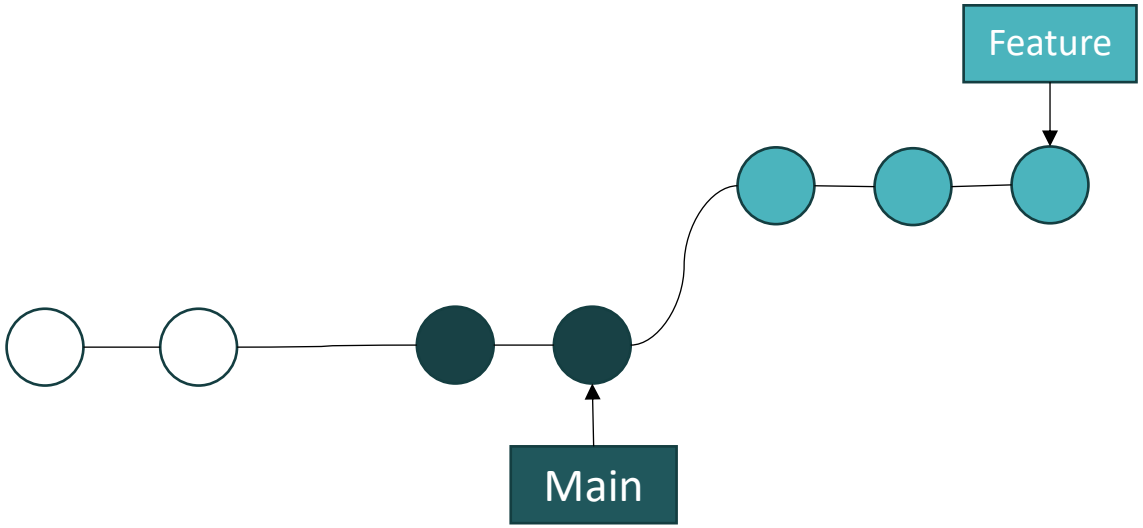




Rebasing

- Rebasing is an alternative to merging.
  - Effectively moves the divergence point to the tip of the Main branch.
- It creates a new commit for each commit on the feature branch.
  - This results in a cleaner project history.
  - However, this action re-writes history - dates and times are changed.
- Golden Rule of Rebasing: Never use it on public branches.





Merge Conflicts

- Most of the time Git can merge changes in the same file on different branches.
- A conflict arises if two branches have changes to the same line of the same file.
  - Or if one branch has deleted the file while another branch edits it.
- Conflicts only affect the developer conducting the merge.
- Git marks the file as being conflicted and halts the merge process.
- Files that are preventing the merge from completing must be manually edited to resolve.

Resolving Merge Conflicts

- The status command will list any merge conflicts.
- The merge option of the log command will give details about merge conflicts.
- The diff function will help find differences between states of a file.
- The abort option of the merge command will exit from the merge process and return to the state before the merge started.

Code Bite

```
git status

git log --merge

git diff

git merge --abort
```

Push Local Repo to Empty Remote

- The push command will copy code from a local repo to a repo on GitHub.
- You must first create the repository on GitHub.
  - Using the web interface is most common approach.
- Then the CLI can be used to push the local repository.
  - First, add a remote origin so the local repo is aware of the remote repo and how to connect.
  - Then push the local repo to the remote location.

Code Bite

```
git remote add origin https://github.com/...


git push origin main
```

Push Command

- Commits to the local branch are committed to the remote branch.
- Fails if it would result in a non-fast-forward merge in the destination.
- Can be forced to occur in the event of a non-fast-forward merge.
  - Remote commits may be lost.
- Commits to all local branches can be pushed.

Code Bite

```
git push


git push origin
git push origin my-feature-branch


git push --force


git push --all
```

Pull Command

- The pull command transfers content from the remote repo to a local repo.
- Commits to the remote branch are committed to the local branch.
  - It will use a merge commit by default.
  - The rebase option will force a rebasing commit instead.

Code Bite

```
git pull


git pull --rebase origin
```

Fetch Command

- The fetch command transfers content from the remote repo to a local repo.
- It does NOT do any kind of merging.
- Changes can then be inspected and merged manually, if desired.
- The pull command is actually a shortcut for fetch and then merge.

Code Bite

```
git fetch


git fetch --all


get fetch origin my-feature-branch
```

Daily Git Workflow - The Full Cycle

```
# Start of day: sync with remote
git checkout main && git pull --rebase origin main

# Create a feature branch
git switch -c feature/MOB-101-time-entry-model

# Work, then stage selectively (patch mode - stage specific hunks)
git add -p src/TimesheetService.swift
git commit -m "feat(time-entry): add TimeEntry model with hours validation"

# Stay current - rebase onto latest main daily
git fetch origin && git rebase origin/main

# Push and open PR
git push -u origin feature/MOB-101-time-entry-model
```

Rebasing - How It Works Internally

```
# Before rebase: your branch diverged from main 3 commits ago
#   main:    A - B - C - D - E  (E is latest)
#   feature: A - B - C - F - G  (F, G are your commits)

# After: git rebase main
#   main:    A - B - C - D - E
#   feature: A - B - C - D - E - F' - G'
#                               ^    ^
#              New commits with same content but different hashes

# F and G still exist in the object database but no branch points to them anymore
# git gc will clean them after 90 days

# GOLDEN RULE: Never rebase commits that other engineers have already based their work on.
```

Working with Remotes

Skill Check

- Find and follow the instructions markdown document in the GitHub repository for the Working with Remotes repo.

Branching Strategies and Conventional Commits

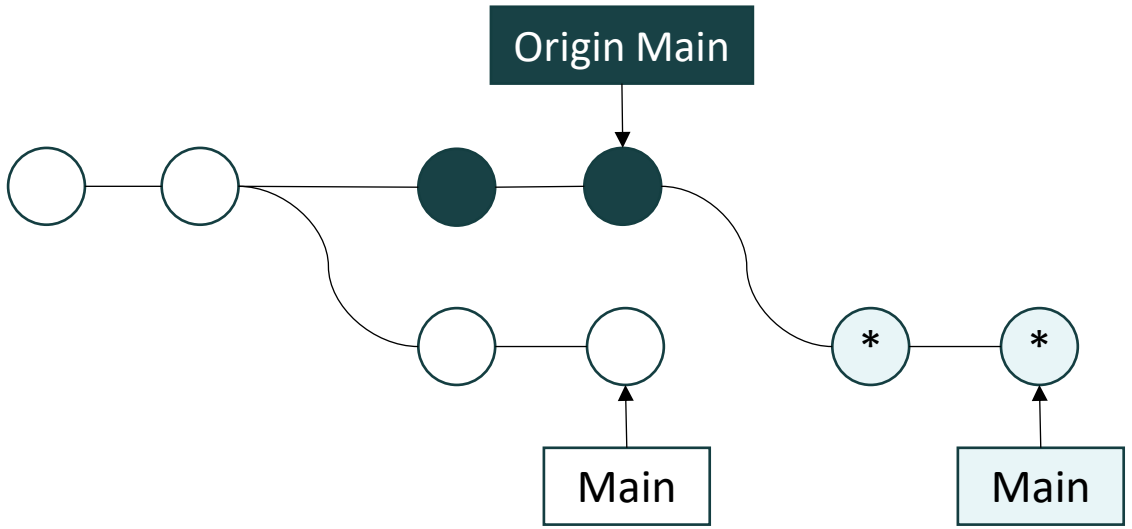
The Centralized Workflow - Ideal Case

- Great for teams transitioning from a centralized VCS.
- Central repository is the single source of truth.
- Uses no branches.

Steps

1. Clone the repository.
2. Make local changes.
3. Commit those changes.
4. When finished, squash commits and push to remote repo.
5. If no changes have been made to remote repo, this will succeed, and you're done.
6. If changes have been made, see below.

Centralized Workflow - Actual Case



Steps

1. Push fails because of changes to remote repo.
2. Pull the origin main branch using the rebase option.
3. Resolve any merge conflicts.
4. Push to remote repo.

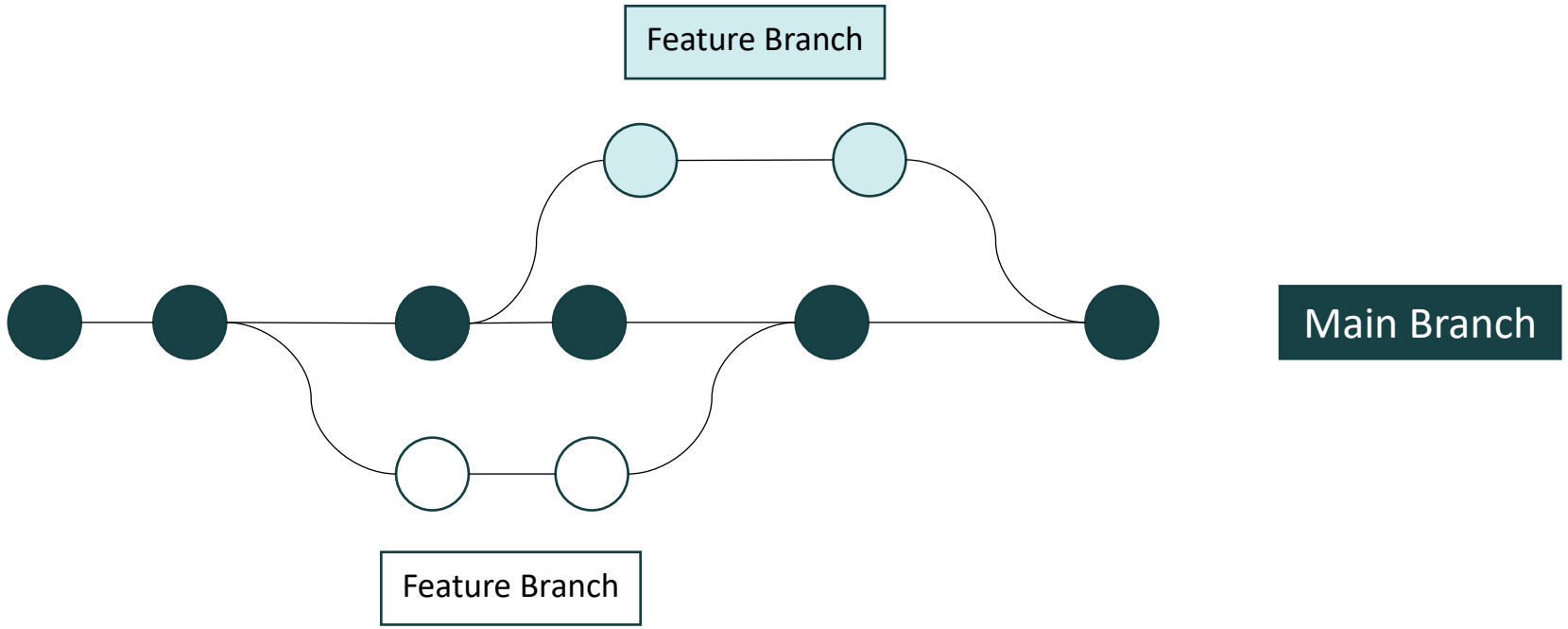
Feature Branching Workflow (GitHub Flow)

- All development takes place on a dedicated branch.
- This ensures Main branch never contains broken code.
- It allows sharing changes through remote repo before they are merged into Main branch.
- This workflow utilizes pull requests.
  - Discussion related to a particular branch.
  - Facilitates code review.
  - Can be started before code is ready for merging.
  - S

Steps

1. Create a branch from main.
2. Develop, commit, repeat.
3. Push branch to remote repo anytime.
4. When done, issue pull request.
5. Other devs, admins review and comment.
6. Fetch remote main.
7. Combine with main using either merge commit or rebasing.
8. Push to remote main.

Feature Branching Workflow Diagram



Trunk-Based Development

- Trunk-based development is a variation of Feature Branching
- It is the practice of all engineers integrating their changes to a single shared branch (the trunk, or main) at least once per day
  - Long-lived feature branches do not exist in TBD
- Feature branches are short-lived: created for a single logical change, reviewed, and merged within hours to 2 days maximum
- Feature flags (feature toggles) control whether new code is visible to users.
  - Code ships to production before the feature is complete; the feature is turned on when it is ready
- CI is mandatory: every push triggers a full build, lint, and test run
  - Failing tests cause the commit to fail
  - Merging broken code to trunk blocks every other engineer
- The trunk is always deployable: any commit on trunk can be shipped to production at any time

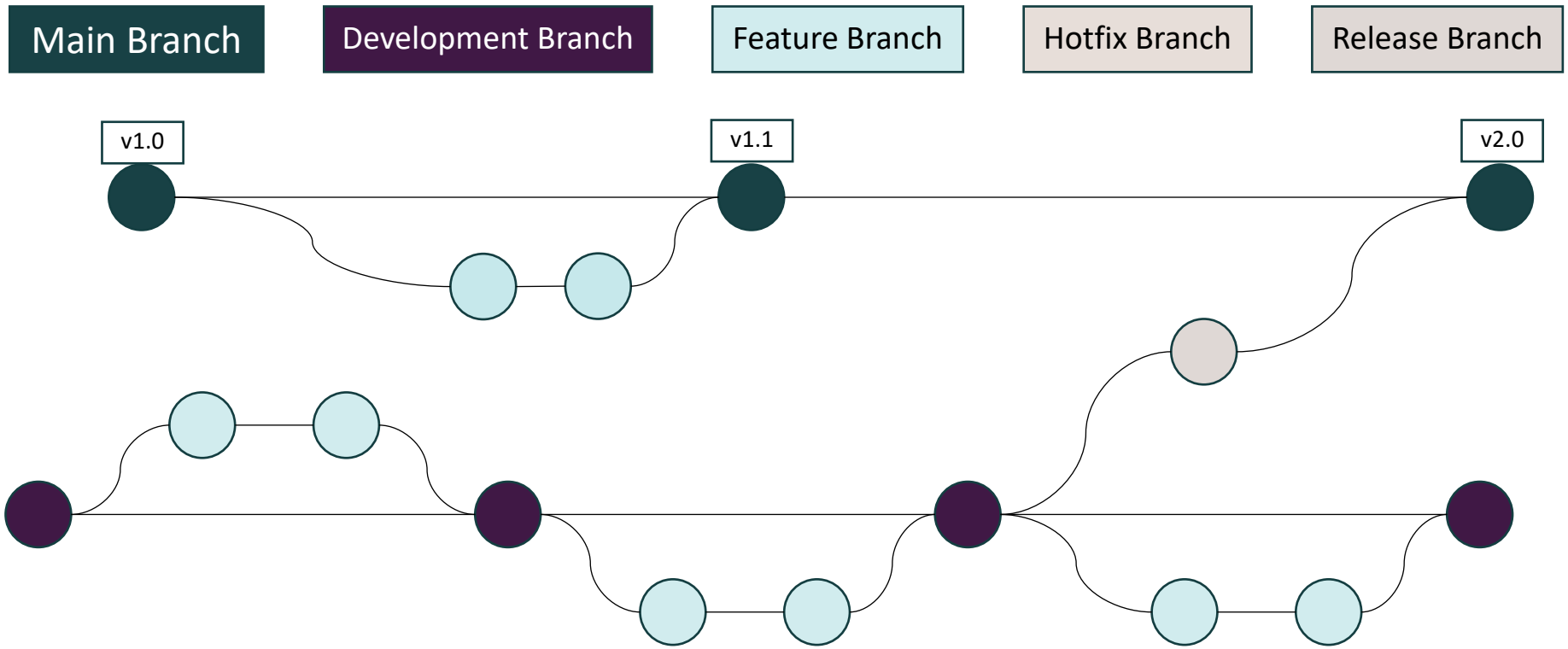
Trunk-Based Development Strengths and Weaknesses

Strengths	Weaknesses
Integration problems are small and frequent (trivial to fix)	Requires disciplined feature flagging - incomplete features must be hidden
Main is always deployable; fastest time-to-production for completed features	Requires a mature CI/CD pipeline; broken builds block everyone
No long-lived branch divergence; merge conflicts are small when they occur	Requires high team trust and code review discipline
Preferred model for companies at Google, Meta, Microsoft scale	Challenging for teams new to automated testing; requires test coverage discipline

GitFlow Workflow

- This workflow is a variation on the Feature Branching Workflow.
  - It has two long-running branches - the Main branch and a development branch.
- Feature branches are split off from and merged to the development branch.
- When releases are imminent, a release branch is split off from the development branch.
  - Once the release branch is QA'd and ready, it is merged into the Main branch.
  - Version numbers are tracked with tags on the merge commits.
- When hotfixes are necessary, a hotfix branch is split off the Main branch.
  - They can be merged back into the Main and the development branch.

GitFlow Workflow



GitFlow Strengths and Weaknesses

Strengths	Weaknesses
Clear structure for teams with formal release schedules	Complex; many long-lived branches; high merge overhead
Well-suited to mobile development with App Store release cycles	Long-lived branches accumulate divergence; integration pain
Separates ongoing development from release stabilization	Slow; features can wait weeks to reach main
Hotfix model is clear and safe	Overkill for teams doing continuous delivery

Forking Workflow

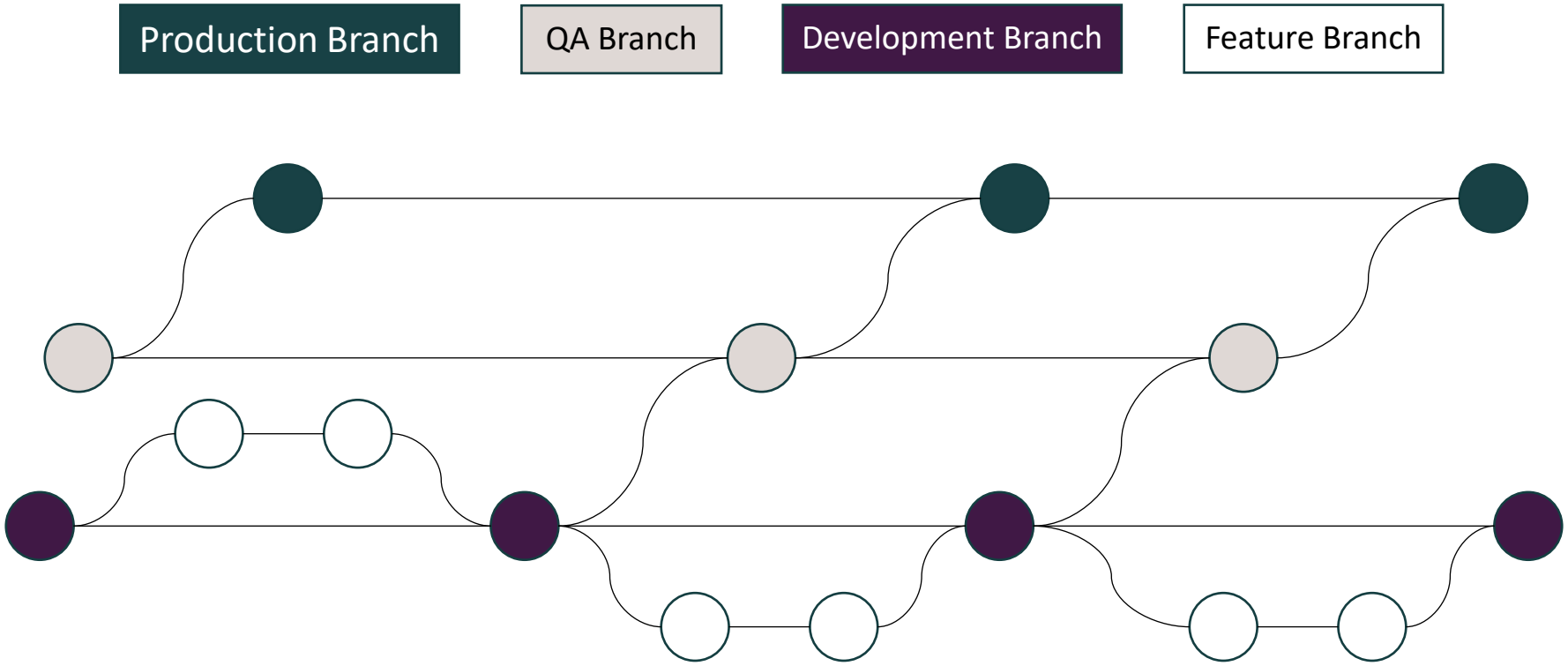
- Many open-source projects work this way.
- It frequently creates multiple versions of the software.
  - E.g. Minecraft builds with different mods
- It provides for many contributors while maintaining centralized control over what code ends up being integrated into the official source.

Steps

1. Each developer forks the official repo and clones it locally.
2. They set up a second remote path for the official repo.
3. Development proceeds on branches and push/merge/rebase onto their own fork.
4. Ultimately issue a pull request from the new branch to the official repo.

Environment Branch Workflow

- Feature branches merge into a development branch.
- A separate long-running branch is maintained for each different environment.



Perils of Long-Running Branches

- Branches work best when they are merged back into the Main branch quickly.
- Problems can arise when your workflow contains multiple long-running branches.
- Without care and discipline, they soon become just a partial subset of the codebase.
  - It can be difficult to know what changes have been merged to which branches and where they have not yet been merged.
- The longer a branch lives before being merged back into Main the more likely it is that merge conflicts will arise.
- Longer-running branches lead to larger chunks of work.
  - That leads to a less responsive and less Agile organizational practice.

Branching Strategies - When to Use Which

Strategy	Key Rule	Requires	Best For
GitFlow	Long-lived main + develop branches. Feature, release, hotfix branches around them.	Discipline to maintain all five branch types.	Teams with scheduled App Store / Play Store release cycles.
Trunk-Based Development	All engineers merge to main at least daily. Feature flags hide incomplete work.	Mature CI/CD pipeline. Feature flag infrastructure. High test coverage.	High-velocity enterprise teams. Continuous delivery.
GitHub Flow	main + short-lived feature branches. PR = the integration gate.	Code review culture. At least 1 approver before merge.	Smaller teams. Simpler delivery model.

Branch Naming Conventions

- Whatever strategy you use, branch names must be consistent across the team. Inconsistent naming produces a repository full of mystery branches that no one can identify or prune safely.

Type	Pattern	Examples
Feature	feature/TICKET-description	feature/MOB-101-time-entry-model, feature/US-001-project-selector
Bug Fix	bugfix/TICKET-description	bugfix/MOB-412-crash-on-nil-employee, bugfix/MOB-389-timezone-offset
Hotfix (prod)	hotfix/TICKET-description	hotfix/MOB-501-auth-token-expiry
Refactor	refactor/TICKET-description	refactor/MOB-380-extract-network-service
Release	release/VERSION	release/2.4.0, release/2025-Q1
Chore	chore/description	chore/update-dependencies, chore/fix-ci-lint-step

Conventional Commits - Format and Examples

```
# Conventional Commits is an industry-standard specification for commit message formats


# Format
<type>(<scope>): <short description>

[optional body - explain WHY, not what - wrap at 72 characters]

[optional footer(s)]
BREAKING CHANGE: <description>
Fixes #<issue-number>
Refs: MOB-412
```

Commit Types

Type	When to Use	Examples
feat	A new feature for the user	feat(auth): add biometric authentication with Face ID fallback
fix	A bug fix for the user	fix(timesheet): resolve nil crash when employee has no submitted hours
refactor	Code change that neither adds a feature nor fixes a bug	refactor(network): extract URLSession into injectable NetworkService protocol
test	Adding or updating tests	test(timesheet): add unit tests for offline sync conflict resolution
docs	Documentation only changes	docs(api): update TimeEntry schema with syncStatus field definition
chore	Build process, dependency updates, configuration	chore(deps): update Alamofire to 5.9.1
perf	A code change that improves performance	perf(timesheet-list): replace O(n^2) duplicate check with Set lookup
ci	Changes to CI configuration files or scripts	ci(lint): add SwiftLint step to GitHub Actions workflow
build	Changes affecting the build system or external dependencies	build: update iOS deployment target to iOS 16
style	Formatting changes that do not affect meaning	style: apply swift-format to all files in Sources/
revert	Reverts a previous commit	revert: feat(auth) add biometric auth - breaking on iOS 15

Conventional Commits Examples

```
# Good examples
feat(auth): add OAuth2 token refresh with automatic retry
fix(timesheet): resolve nil crash when employee has no submitted hours
refactor(network): extract URLSession into injectable NetworkService protocol
test(sync): add unit tests for offline conflict resolution - MOB-412

# Never
git commit -m "stuff"
git commit -m "fix bug"
git commit -m "WIP"
git commit -m "JIRA-1234"
git commit -m "fixed the thing"
```

Branching Strategies

Skill Check

- Find and follow the instructions markdown document in the labs GitHub repository for the Branching Strategies folder.

Pull Requests and Code Review

Pull Requests

- Pull requests let you tell others about the changes you have pushed to a branch.
- You can discuss the changes with collaborators.
- You can add follow-up commits as development proceeds.
- After everyone is satisfied with the changes, the pull request can be merged into the main branch.
- Pull requests serve three purposes:
  - a quality gate
  - a knowledge transfer mechanism
  - a paper trail for decision making
- Pull requests must have quality descriptions
  - A PR with no description forces reviewers to read every line of code to understand the context, intent, and testing approach

Anatomy of a Pull Request

```
## Title
feat(offline-sync): implement offline time logging with Core Data / Room persistence

## What this PR does
Implements the offline time logging capability described in US-005.
Field technicians can now log hours in areas with no connectivity.
Entries are stored locally using Core Data (iOS) and Room (Android)
and sync automatically when connectivity is restored.

## Motivation and context
This is the first part of the offline capability. US-005 has been split:
- This PR: local storage and sync trigger
- MOB-510: conflict resolution UI (separate PR, next Sprint)

## Changes included
- Added LocalTimesheetStore wrapping Core Data / Room
- Added SyncEngine that observes network reachability
- Added SyncStatus enum (.synced, .pendingSync, .conflicted)
- Updated TimeEntryViewModel to write to local store when offline
- Added 'Pending Sync' label to TimeEntryCell when syncStatus is .pendingSync
```

Anatomy of a Pull Request - continued

```
## Linked ticket
Closes MOB-412 (main ticket)
Related: MOB-510 (conflict resolution, not in this PR)

## Testing
Unit tests: TimesheetSyncServiceTests - 14 tests, all passing
Manual testing:
  - Enabled airplane mode, logged 3 entries - saved with Pending Sync label ✓
  - Re-enabled WiFi - entries synced within 5 seconds ✓
  - Tested on iPhone 14 Pro (iOS 17.2) and Pixel 7 (Android 14)

## Screenshots
[Attach: offline mode indicator, pending sync label, sync success notification]

## Review checklist
- [x] Tests written and passing
- [x] No linting errors (SwiftLint / ktlint all clear)
- [x] Reviewed against acceptance criteria from US-005
- [x] API changes documented in TimeTracking API Reference v2.3
- [ ] Performance testing with large offline entry set (> 100 entries) - TODO in MOB-515
```

Pull Request Size and Scope

- Large pull requests are a maintenance and quality problem. Code review quality degrades exponentially as PR size increases. Studies show that reviewers catch far fewer defects in PRs with 400+ lines of changes than in PRs under 200 lines.

PR Size	Characteristics	Review Quality	Action
Small (< 150 lines)	One logical change, focused scope, easy to understand in one sitting	High - reviewers can hold the entire change in working memory	Ideal. Aim for this.
Medium (150-300 lines)	A feature with a few components; a larger refactor	Acceptable - reviewers may need to re-read sections	Acceptable with a clear description.
Large (300-600 lines)	Multiple logical changes bundled together; or a feature with UI, logic, and tests all in one PR	Degraded - reviewers skim; defects slip through	Split if possible. Separate UI from logic, or separate tests from implementation.
Very Large (600+ lines)	Generated code, database migrations, major feature, or multiple features	Poor - reviewers rubber-stamp; review is theatrical, not useful	Must be split. Use draft PRs and stacked PRs to break into reviewable increments.

Creating a Pull Request

- Anyone with read permissions on a repository can create a pull request.
- You must have write permissions to create a branch, however.
- You cannot create a pull request to merge a branch into itself.
- The base branch is where you want your changes to be applied.
- The compare branch contains the changes you would like to be applied.

Steps

1. Navigate to the main page of the repository and select the Pull Requests tab.
2. Select New pull request.
3. Use the "base" dropdown to select the target branch and the "compare" dropdown to select your topic branch.
4. Type a title and description for your pull request.
5. Select Create pull request.

Code Review Comments

- Code review is a professional skill, not a gatekeeping exercise
- Effective reviewers make specific, actionable, constructive comments
  - They distinguish between required changes and optional suggestions
  - They recognize good work explicitly

Comment on the code, never on the person. 'This variable name is confusing' is a code comment. 'You always name things badly' is a personal attack. The difference seems obvious in the abstract but is easy to blur when you are under deadline pressure. Every comment you write is a public record that your teammates and future colleagues will read.

Code Review Comment Types

Type	Prefix	When to Use	Example
Blocking	[BLOCKING]	Defect, security risk, correctness problem. Must be resolved before merge.	[BLOCKING] Force-casting as String crashes on null. Use safe optional binding.
Non-blocking	[NIT]	Style improvement, naming, alternative worth considering. Not worth blocking.	[NIT] syncPendingEntries() is clearer than processPendingSync() per our verb-noun convention.
Question	[QUESTION]	Don't understand the reasoning. May be a bug or may have a good reason.	[QUESTION] Why retry interval doubling instead of fixed backoff? Performance reason?
Positive	(no label)	Explicitly recognize good engineering. Do this often.	Clean use of the Strangler pattern - the new sync engine plugs in without touching existing persistence code.

Completing a Pull Request

- The pull request can be completed three ways:
  - Creating a merge commit to merge all the compare branch commits onto the base branch.
  - Squashing all the compare branch commits into one commit which is then merged onto the base branch.
  - Rebasing all the compare branch commits onto the base branch and then applying a fast-forward merge.

Steps

1. In your repository select Pull Requests.
2. Select the pull request you wish to merge.
3. Select either the merge, squash or rebase option.
4. Type a commit message.
5. Select the Confirm button.
6. Optionally, delete the branch.

Merge Conflicts and Advanced Git

Merge Conflicts

- Merge conflicts are not failures
- They are a normal, expected consequence of multiple engineers working on the same codebase simultaneously
- Healthy collaboration produces occasional conflicts
  - The skill is in resolving them quickly and correctly
- Git automatically merges changes when they are to different parts of the same file, or to entirely different files. A conflict occurs when:
  - Two branches have modified the same line(s) in the same file.
  - One branch has modified a file that the other branch has deleted.
  - Both branches have added a file at the same path with different contents.

Anatomy of a Conflict Marker (diff3 style)

```
<<<<<<< HEAD  (your branch - current)
func fetchTimesheets(for employeeId: String, limit: Int = 20) -> [Timesheet] {
    return repository.fetchTimesheets(employeeId: employeeId, limit: limit)
}
||||||| merged common ancestors  (the original - both branches started here)
func fetchTimesheets(for employeeId: String) -> [Timesheet] {
    return repository.fetchTimesheets(employeeId: employeeId)
}
=======  (incoming change - being merged in)
func fetchTimesheets(for employeeId: String) -> [Timesheet] {
    return repository.fetchTimesheets(employeeId: employeeId, limit: 50)
}
>>>>>>> feature/US-002-manager-view

# Enable diff3: git config --global merge.conflictstyle diff3
```

The 7-Step Conflict Resolution Process

1. Stop. Read BOTH versions and the common ancestor. Understand each engineer's intent.
2. Identify the correct resolution - is one version right? Do both need to coexist? Is the right answer a synthesis?
3. Write the resolution - edit the file to the correct merged result. Remove ALL conflict markers.
4. Verify - the file must compile. Run the existing tests. The resolution must be correct, not just syntactically valid.
5. Stage the resolved file: git add <filename>
6. Complete: `git merge --continue` OR `git rebase --continue`
7. Write a meaningful commit message - explain the resolution decision, especially if you chose one side over the other.

Preventing Conflicts

- Not all conflicts can be prevented, but their frequency can be reduced

Practice	How It Reduces Conflicts
Integrate frequently	The longer a branch lives, the more divergence accumulates. Rebasing onto main daily means you see others' changes before they compound.
Small, focused branches	A branch that touches 3 files conflicts with a smaller surface area than a branch that touches 30.
Communicate before touching shared files	If two engineers both plan to refactor the same service, coordinate: one goes first, the other rebases after.
Decompose work to reduce overlap	User story breakdown that assigns clear ownership over specific layers or modules reduces the likelihood of two engineers editing the same file.
Consistent code formatting	Automated formatting (swift-format, ktlint) means that a formatting change never produces a conflict. Without automation, one engineer's formatting preferences conflict with another's.

Amending Commits

- The amend option of the commit command replaces the most recent commit with a new one.
- If executed after staging additional changes, it will combine them into the most recent commit.
- If executed before staging additional changed, it can be used to modify the commit message of the most recent commit.

Code Bite

```
git commit --amend

git commit --amend -m "modified message"
```

Interactive Rebasing

- Interactive rebasing allows you to go back in time and replay a sequence of commits, making changes to the way they were applied.
- The -i option initiates interactive rebasing.
- You will need to specify the commit that will serve as the starting point.
  - This is basically the last commit you wish to keep as-is.
- You can reference it by its SHA-1 hash, or by a relative reference from HEAD.
  - The relative reference uses the ~ to specify the index.

Code Bite

```
git rebase -i 528f82e

git rebase -i HEAD~3
```

Interactive Rebasing Options

- You have several options that can be applied to each commit being replayed:
  - Pick - apply the commit unchanged.
  - Squash - combine the commit with the previous commit.
  - Fixup - combine the commit with the previous commit.
  - Reword - keep the commit as-is but change the commit message.
  - Edit - pause the rebase on this commit for manual editing before continuing.
  - Drop - omit the commit, losing its changes forever.
- You get to specify which option to apply to each commit that occurred after the one specified as the starting point.
- Once the options are specified, interactive rebasing will run through the commits using the specified options and re-write history.
  - NOTE: the original commits will be replaced with new commits, complete with new SHA-1 hashes.

Combining Commits

- Most developers make frequent commits while developing features.
  - This is helpful during development, giving a lot of opportunity for reverting in case something goes drastically wrong.
  - However, this also creates a complex history to unravel later.
  - Once the feature is fully developed, that granular opportunity to roll back is no longer needed.
- Before synchronizing or creating a pull request, you may wish to simplify that history.
- The Squash and Fixup commands both combine commits.
  - Squash combines the commits and their messages, allowing the combination to be edited.
  - Fixup combines the commits and retains only the previous commit's message.

Cherry-Picking

- The *cherry-pick* command allows arbitrary commits to be selected from another branch and applied to the current HEAD.
  - Useful for things like bug fixes developed on a feature branch also being applied to Main for production deployment.
  - Useful for applying to Main some of the changes from a feature branch that is being abandoned.
- The -e option allows you to edit the commit message first.
- The -n option will apply the changes to your working copy but not commit them.

Code Bite

```
git cherry-pick 89cd503

git cherry-pick -e fa3a43b

git cherry-pick -n 92b22cf
```

Bisect

- The *bisect* command starts a wizard that iteratively checks out commits in order to narrow down where a bug was introduced.
- It requires specification of the last known good commit and the first known bad commit.
- It will keep checking out commits and waiting to be told if they are good or bad until it narrows down the offending commit.

Code Bite

```
git bisect start
git bisect good 45e8279
git bisect bad b6c73a1
<check code>
git bisect bad
<check code>
git bisect good
<check code>
git bisect good
<etc.>
```

Advanced Git - Your Safety Net: The Reflog

```
# The reflog records every time HEAD or a branch reference moved
git reflog
# abc1234 HEAD@{0}: commit: feat(sync): implement sync engine
# def5678 HEAD@{1}: rebase (finish): returning to feature/offline-sync
# jkl3456 HEAD@{3}: checkout: moving from main to feature/offline-sync

# Scenario: You accidentally ran 'git reset --hard HEAD~5'
# and lost 5 commits. They're not gone.
git reflog
# Find HEAD@{1} - the state BEFORE the reset
git checkout -b recovery-branch HEAD@{1}
# Your lost commits are on recovery-branch

# Reflog retains entries for 90 days.
# Almost any Git mistake is recoverable within that window.
```

Tags

- You can tag specific points in a repository's history as being important.
  - Their most common use is to mark release points (v1.3, v2.0, etc.).
- You can create two types of tags - lightweight and annotated.
- Like a branch, a lightweight tag is just a pointer to a commit.
- An annotated tag is stored as a full object in the Git database.
  - It is checksummed and contains tagger name, email and date.
  - It can be digitally signed.
  - This is the recommended type of tag to use.

Listing and Creating Tags

- The *tag* command on its own (or with the -l option) will list your tags.
- Providing a tag name with the command will create a lightweight tag.
  - The tag will be applied to the commit currently referenced by HEAD.
- Adding the -a option with a tag name will create an annotated tag.
  - The -m option will allow addition of a message to the tag.
- Adding a commit has will apply the tag to a commit other than the one referenced by HEAD.

Code Bite

```
git tag
git tag -l


git tag v1.3


git tag -a v1.4 -m "November policy update"


git tag -a v1.2 9fceb02
```

Sharing and Removing Tags

- Cloning and pulling will copy any existing tags along with the repo.
- Pushing does NOT copy tags, by default.
- Listing a tag name after the *push* command will push just that tag.
- Using the *push* command with --tags option will push all tags.
- Using the *tag* command's -d option will delete a tag.
  - It is only deleted locally.
- Use the *push* command with --delete option to remove the tag in a remote repo.

Code Bite

```
git push origin v1.4

git push origin --tags



git tag -d v1.3

git push origin --delete v1.3
```

Merge Conflicts and Advanced Git

Skill Check

- Find and follow the instructions markdown document in the GitHub repository for the Merge Conflicts Advanced Git folder.

CI/CD Integration

How CI Interacts with Your Git Workflow

Git Event	CI Trigger	What Runs
Push to any branch	Branch push	Build · Lint · Unit tests · Status on commit
PR opened or updated	PR trigger	Full build + lint + unit + integration tests. Must pass before merge.
Merge to main	Main push	Full suite + coverage check + security scan. Auto-deploy to TestFlight/Firebase.
Tag push (release)	Tag trigger	Build release artifact (IPA/APK) · Submit to App Store / Play Store

Branch Protections

- Protected branches allow you to set preconditions before pull requests can be merged.
  - By default, any pull request can be merged at any time, provided there are no merge conflicts.
- Protected branches are available for all public repositories.
  - They are also available for private repositories with GitHub Pro, GitHub Team, GitHub Enterprise, GitHub Enterprise Cloud, and GitHub Enterprise Server.
- Repository Owners and people with admin permissions can manage branch rules.
- Branch rules can apply to a specific branch, all branches in the repo, or only branches that match a specified naming pattern.
- The existence of a branch rule will prevent collaborators from force pushing or deleting the protected branch.

Configuring Protected Branches

- Branches can be configured to require:
  - A minimum # of pull request reviews before merging.
  - A selection of status checks to pass before merging.
  - Digitally signed commits.
  - A linear history.
  - A limited pool of users who can push to the branch.
- In addition, you can choose to allow force pushes and/or deletions.
  - Both are disallowed by default.

Steps

1. Navigate to the repository's main page.
2. Select Settings.
3. In the left menu, select Branches.
4. Select Add rule.
5. Enter the branch name or pattern you wish to protect.
6. Select the desired branch rule settings.
7. Select Create.

Enabling Required Status Checks

- Status checks are based on external processes, such as continuous integration builds.
- They are created through GitHub's API.
  - Details can be found in the GitHub Developer documentation.

Steps

1. Navigate to the repository's main page.
2. Select Settings.
3. In the left menu, select Branches.
4. Select Add rule.
5. Enter the branch name or pattern you wish to protect.
6. Select "Require status checks to pass before merging"
7. Select the checks you want to require.
8. Select Create.

Enabling Branch Restrictions

- Branch restrictions limit the users, teams or apps who can push to a protected branch.

Steps

1. Navigate to the repository's main page.
2. Select Settings.
3. In the left menu, select Branches.
4. Select Add rule.
5. Enter the branch name or pattern you wish to protect.
6. Select "Restrict who can push to matching branches".
7. Select the users, teams or apps desired.
8. Select Create.

Branch Protection Rules - Enforce Your Standards

Rule	What It Prevents
Required status checks (CI must pass)	Broken builds cannot be merged - not even under deadline pressure
Required reviews (1-2 approvals)	Code ships without a second set of eyes
Dismiss stale reviews on new push	Post-approval code changes that introduce bugs slipping through
Require conversation resolution	Open review comments being silently ignored on merge
Restrict force pushes to main	Accidental or intentional history rewriting on shared branches

Knowledge Check

Knowledge Check · Question 1

A senior engineer tells a junior: 'Just push directly to main - we'll skip the PR for this one because we're in a rush.' As the Scrum Master, how do you respond?

Knowledge Check · Question 2

Explain the difference between git fetch and git pull. Under what circumstances would you prefer each?

Knowledge Check · Question 3

What does the Conventional Commits specification produce beyond consistent commit message formatting?

Knowledge Check · Question 4

A production crash is reported. The bug was introduced sometime in the last 100 commits. What Git operation finds it in ~7 steps instead of reading all 100?

Knowledge Check · Question 5

You force-pushed to main by mistake, rewriting the last 3 commits. Three engineers have already pulled those commits. Describe the exact recovery steps.

PHASE 01: SOFTWARE ENGINEERING FOUNDATIONS FOR MOBILE

Problem Solving and Engineering Thinking

Module 3

Requirement Decomposition

THE TWO FAILURE MODES

Underestimate: you only see the happy path. Over-engineer: you see the complexity but have no framework to prioritize it.

Decomposition is the skill that navigates between them.

Origin of Problems

- Most software defects, schedule overruns, and architectural misfires originate from
  - Misunderstood requirements
  - Requirements that were understood but never broken into work small enough to execute with clarity
- Decomposition is the skill of breaking work into small and clear enough parts

Unactionable

Users should be able to manage their time off

Actionable

Implement PATCH/time-off-requests/:id/status endpoint, accepting approved or denied, returning 200 with updated resource or 409 on invalid transition, with integration tests covering both status codes

Decomposition

- Decomposition requires
  - Understanding the problem domain deeply enough to see all the pieces
  - Understanding the technical architecture well enough to know how those pieces relate
  - Understanding the team well enough to know what can be parallelized and what cannot
- Engineers who cannot decompose work effectively produce one of two failure modes:
  - They underestimate because they see only the happy path
  - They over-engineer because they see the full complexity without a framework for prioritizing
- Both outcomes are damaging

The Five-Step Decomposition Framework

Step	Action	Key Question
1. Read	Understand the user GOAL, not just the stated feature	What is the user trying to accomplish? What is the business value?
2. Identify Components	Enumerate ALL layers: data, persistence, API, service, ViewModel, UI, navigation, tests, analytics, accessibility	What data structures, screens, and services does this feature touch?
3. Map Dependencies	Identify what must exist before each task can start. Find the critical path.	What blocks what? What can run in parallel?
4. Write Tasks	Each task: one engineer, under half a day, specific completion criteria	Can anyone on the team read this and know exactly what to do?
5. Identify Unknowns	Name anything that requires investigation before estimation. Define a spike.	What do we NOT know yet that would change this estimate significantly?

Step 1: Understand the Goal

- Don't just read the requirement - ask questions of the stakeholders
  - The most important question is "why?"

As Written	The Goal Behind It	Why the Distinction Matters
'Add a search bar to the timesheet list'	Managers need to find a specific employee's timesheet quickly without scrolling through a long list	If the list only ever has 8 entries, a search bar adds no value. The goal might be better served by alphabetical sorting or a filter. Feature-first thinking builds unnecessary work.
'We need push notifications'	Field technicians forget to submit timesheets and miss the Friday payroll deadline	Push notifications are one solution. An in-app badge on the app icon is another. A persistent banner on the home screen is another. Understanding the goal opens the solution space.
'Build an offline mode'	Field technicians work in areas with no cellular connectivity and lose work when they try to log hours', which causes payroll errors and resubmission overhead	The goal clarifies scope: we need to preserve work done offline, not rebuild the entire app for offline-first architecture. That is a much smaller problem.

Step 2: Identify the Components

Layer	Questions to Ask
Data models	What new data structures does this feature require? Do any existing models need new fields? What are the data types? Are there validation rules?
Persistence	Does this data need to be stored locally (Core Data / Room)? Does it need to survive app restarts? What happens to existing local data if the schema changes?
API contracts	What new endpoints are needed? What are the request and response schemas? What HTTP status codes are expected? Who owns the backend implementation - the mobile team or a separate backend team?
Service / business logic	What logic runs between the UI and the data layer? What are the business rules (validation, state transitions, calculations)?
ViewModel / state	How does the UI state change in response to this feature? What loading, success, and error states does the UI need to render?
UI / screens	How many new screens or components are needed? What does each state look like (loading, empty, error, populated)? Are there animations?
Navigation	Does this feature require new navigation routes? Are there deep links from notifications or other apps?
Testing	Unit tests for service logic. Integration tests for data layer. UI tests for critical paths. What are the edge cases (empty state, zero, maximum values, network failure)?
Analytics / logging	What events need to be tracked for product metrics? What diagnostic information needs to be logged for engineering?
Accessibility	Does the feature require VoiceOver / TalkBack support? Are there dynamic type considerations? Are interactive elements large enough to tap?

Step 3: Map Dependencies

- A dependency is a task that must be finished before another task can start
- Dependency mapping lists out each task and what its dependencies are
- Mapping reveals the critical path; i.e. the longest chain of sequential dependencies
  - This determines the minimum possible delivery time regardless of team size

Dependency Example: Offline Time Logging

Task A: Define TimeEntry data model (fields, types, validation rules)
  --> BLOCKS: Task B, Task C, Task E

Task B: Design API contract for time entry endpoints (POST, GET, PATCH)
  --> BLOCKS: Task D, Task F
  --> REQUIRES: Task A (model must be defined before API schema)

Task C: Implement LocalTimesheetStore (Core Data / Room)
  --> BLOCKS: Task G, Task H
  --> REQUIRES: Task A (model must be defined before schema)

Task D: Implement backend API endpoints (backend team)
  --> BLOCKS: Task F (mobile cannot integrate until API is live/mocked)
  --> REQUIRES: Task B

Task E: Build time entry UI (project selector, hours input, date picker)
  --> BLOCKS: Task G
  --> REQUIRES: Task A (needs model for binding)

Dependency Example: Offline Time Logging

Task F: Implement TimesheetService API integration (online path)
  --> BLOCKS: Task H (sync engine needs both local and remote service)
  --> REQUIRES: Task B, Task D (or mock)

Task G: Connect UI to LocalTimesheetStore (offline write path)
  --> BLOCKS: Task I
  --> REQUIRES: Task C, Task E

Task H: Implement SyncEngine (detect connectivity, sync pending, handle conflicts)
  --> BLOCKS: Task I
  --> REQUIRES: Task C, Task F

Task I: Integration testing and manual QA
  --> REQUIRES: Task G, Task H

Critical Path

- Critical Paths:
  - A -> C -> G -> I
  - A -> B -> D -> F -> H -> I
- Can parallelize:
  - Task E and Task C (after A is done)
  - Task D and Task C

Step 4: Write Concrete, Estimable Tasks

- Each task must be completable by one engineer in a known timeframe
  - Should fit within one day or less
  - Should be SMART

Quality	Question to Ask	Bad Task	Good Task
Specific	Can anyone on the team read this and know exactly what to do?	'Implement data model'	'Define Swift struct TimeEntry with fields: id (UUID), employeeId (String), date (Date), hours (Double, clamped 0.25-24.0), projectCode (String), syncStatus (SyncStatus enum)'
Measurable	How do we know this is done?	'Work on the sync engine'	'SyncEngine.processPendingEntries() makes POST /time-entries for each .pendingSync entry; updates status to .synced on 201; leaves as .pendingSync on network error; sets .conflicted on 409'
Assigned	Is there a named owner?	(unassigned)	'Maria: implement iOS offline storage using Core Data'
Realistic	Can this be done by one person in less than a day?	'Build the entire offline capability'	Split into 6-8 tasks each under half a day
Testable	Is there a test that will tell us if this is done correctly?	'Add error handling'	'fetchTimesheets() returns Result.failure(.networkTimeout) when URLSession times out; unit test confirms error type without network'

Step 5: Identify Unknowns - Technical Spikes

- A technical spike is a time-boxed investigation into an unknown that prevents the team from estimating a story with confidence
- Spikes are legitimate backlog items
  - They are not failures of preparation
  - They are the mechanism Agile provides for dealing with genuine uncertainty.
- When to create a spike:
  - The team has never integrated with this third-party SDK, API, or platform feature before
  - There is significant disagreement in planning poker estimates (e.g., votes of 3 and 13), indicating different team members have very different mental models of the complexity
  - A technical assumption is central to the estimate but unverified (e.g., 'we assume biometric auth works on all company-issued devices')
  - The architecture of the solution is unclear - multiple approaches are plausible and the right choice is not obvious without prototyping

What a Spike Looks Like

```
SPIKE: MOB-SPIKE-012
Title: Investigate background sync capability on iOS and Android

Context:
US-005 (offline sync) requires the app to sync pending entries when
connectivity is restored, even if the app is in the background.
We do not know which background task APIs are available to us or
whether our enterprise MDM configuration restricts background execution.

Questions to answer:
1. Does iOS BackgroundTasks framework (BGAppRefreshTask) work with
   our enterprise MDM profile?
2. Does WorkManager on Android schedule tasks reliably on all
   company-issued Samsung devices (some manufacturers limit background work)?
3. What is the maximum execution time allocated for background tasks?
4. Can we trigger a sync from a push notification (background content push)?
5. What are the battery usage implications?

Time-box: 4 hours (half a day)
Owner: Maria (iOS) and Dev (Android), run in parallel

Output required:
- Written answers to each question above
- A revised estimate for US-005 based on findings
- A recommended technical approach (or two with tradeoffs)
- Any new risks or dependencies identified

Definition of Done for the spike:
Team is confident enough to estimate US-005 at the end of this spike.
If not, the spike surfaces a new spike or a decision needed from stakeholders.
```

Dashboard Decomposition Challenge

Skill Check

- Find and follow the instructions markdown document in the labs GitHub repository for the Dashboard Decomposition folder.

Technical Communication

Technical Communication

- Writing clearly about technical problems is a non-optional professional skill
- Engineers who communicate precisely move faster:
  - They spend less time in clarification loops
  - They build more trust with stakeholders
  - They produce better documentation for future team members.

The Problem Statement

- A well-formed problem statement answers four questions
- Every question must be answered - missing one leaves a gap that forces the reader to make assumptions

Question	What It Provides	What Happens Without It
What is happening?	The observed behavior - what the system is doing	The reader must guess the starting point
What should be happening?	The expected behavior - what correct behavior looks like	Without a clear expected state, 'wrong' has no definition
How do you reproduce it?	The exact steps to trigger the problem	Without reproduction steps, the bug cannot be investigated or verified as fixed
What is the impact?	Who is affected, how severely, and at what frequency	Without impact, the bug cannot be prioritized correctly

Problem Statement Examples

UNACCEPTABLE: 'The app is broken for managers.'

  Missing: what breaks, when it breaks, how to reproduce it,
  what the correct behavior is, and who is affected.
  This statement is a complaint, not a problem description.

POOR: 'The timesheet list crashes sometimes.'

  Partially identifies the symptom (crash) and location (timesheet list).
  Missing: when 'sometimes' is (conditions, frequency), reproduction steps,
  expected behavior, and impact. Cannot be investigated.

ACCEPTABLE: 'The timesheet list screen crashes for managers on Monday mornings.'

  Identifies symptom, location, actor, and timing.
  Missing: reproduction steps, expected behavior, impact scope.
  An engineer could start investigating but might not be able to reproduce it.

Problem Statement Examples

GOOD:

'The timesheet list screen crashes with EXC_BAD_INSTRUCTION when a manager opens the Timesheets tab before any employee on their team has submitted hours for the current week. The app should display an empty state view with the message 'No timesheets to review this week.'

To reproduce:
1. Log in as manager@company.com (test credentials in 1Password)
2. Ensure no employees on the manager's team have submitted timesheets for the current week (Monday-Sunday)
3. Tap the Timesheets tab
4. App crashes immediately

Impact: All managers are completely blocked from reviewing timesheets at the start of each week. Affects approximately 45 managers across all regional offices. Bug was introduced in build 2.4.1 (released 2025-06-10). Crash report: Crashlytics ID CL-20250610-0084'

Why this is good: A developer who has never seen this bug can pick up this description and reproduce, investigate, and verify a fix independently.

Writing for Mixed Technical and Non-Technical Audiences

- Most engineering communication crosses audience boundaries
- A bug report goes to QA (technical), the Product Owner (domain expert), and sometimes a client or executive (neither)
- A deployment plan goes to engineering (technical) and operations (semi-technical) and finance (non-technical)
- The same facts must land clearly for all readers.

Principles for Technical Writing for Mixed Audiences

- Lead with impact, follow with cause. 'Managers cannot access timesheets on Monday mornings' is understood by everyone. 'The app force-unwraps an Optional that is nil when the timesheet array is empty' is for engineers only. Put the impact first.
- Define technical terms the first time. 'A crash (an unexpected application exit that requires the user to relaunch)' is one parenthetical that makes the entire message accessible.
- Separate the summary from the detail. A two-paragraph summary followed by a technical appendix serves everyone: executives read the summary, engineers read both.
- Never use technical jargon as precision cover. 'The API returned a 503' does not tell a non-technical reader anything. 'The server was temporarily unavailable, causing the app to display an error instead of loading timesheets' does.
- Use active voice. Passive voice obscures ownership. 'A fix was deployed' is weaker than 'The iOS team deployed a fix to TestFlight at 2:15pm and it is awaiting QA verification.'

Incident Reports

- When a production bug causes significant user impact, an incident report documents:
  - What happened
  - The timeline of investigation and response
  - The root cause
  - The steps taken to prevent recurrence
- Learning to write these well is a senior engineering skill

Incident Report Example - part 1

```
INCIDENT REPORT — MOB-INC-2025-047
Date: 2025-06-16
Severity: SEV-2 (significant user impact; no data loss)
Duration: 4 hours 12 minutes
Author: Maria Chen
Reviewed by: Alex Kim (Engineering Lead)

SUMMARY
The iOS TimeTracking app v2.4.1 crashed immediately on launch for all users
who had the app in the background during an OS push notification delivery
between 08:00 and 12:12 on 2025-06-16. Approximately 340 users were affected.
The issue was caused by a nil reference in the push notification handler
introduced in the 2.4.1 release.

TIMELINE
08:00 — Scheduled push notification sent to all users re: timesheet deadline
08:03 — Crashlytics alert fires: crash rate exceeds 0.1% threshold
08:07 — On-call engineer Maria Chen acknowledges alert
08:15 — Crash traced to NotificationHandler.handleTimesheetReminder()
08:30 — Root cause identified: userInfo["timesheetId"] cast to String! without guard
08:45 — Fix committed and pushed to TestFlight
10:30 — QA verifies fix on TestFlight build
11:00 — App Store expedited review requested
12:12 — Fix released as v2.4.2
```

Incident Report Example - part 2

```
ROOT CAUSE
The notification payload was updated to include a new 'timesheetId' field in
the push notification backend. The iOS handler was not updated simultaneously.
When 'timesheetId' was absent from the payload (which it was for all users
who had not yet submitted a timesheet), the force-cast to String! produced
EXC_BAD_INSTRUCTION.

CONTRIBUTING FACTORS
1. No integration test for notification handler with missing payload fields
2. Backend and mobile teams released simultaneously without integration testing
3. The notification handler was the only class in the codebase still using
   force-cast (as!) for dictionary access; this was a known technical debt item
   that was not yet addressed

REMEDIATION (completed)
1. Replace all force-casts in NotificationHandler with safe optional binding
2. Add unit tests for all notification handler paths including missing fields
3. Add a contract test: backend push payload schema is validated against the
   iOS handler's expected schema on every backend release

PREVENTION (to be completed by 2025-06-30)
4. Add SwiftLint rule: no 'as!' in notification handlers
5. Add 'coordinated release checklist' to process when backend + mobile
   release together: requires integration test sign-off from both teams
6. Add 'notification payload schema' to API documentation and version it
```

Systematic Debugging Methodology

TWO KINDS OF DEBUGGERS

Heroic: stares at code, changes things randomly, eventually fixes it through intuition and persistence.

Systematic: forms a hypothesis before making a change. Tests one variable at a time. Understands the failure before fixing it.

Types of Debugging

- The heroic debugger stares at code until something looks wrong, changes things randomly, and eventually fixes the bug through a combination of intuition and persistence
- They are celebrated when they succeed, but the process is slow, non-reproducible, and teaches nothing. The same bug pattern will recur.
- The systematic debugger follows a framework
  - They form a hypothesis before making a change
  - They test one variable at a time
  - They verify their understanding of the failure before they fix it
  - They are slower at the beginning of the investigation and dramatically faster at the end
  - Because they understand why the bug occurred, they can prevent the same pattern from recurring
- Professional teams expect systematic debugging
- A developer who says 'I'm not sure why, but it works now' has not fixed a bug - they have papered over a symptom

The 7-Step Debugging Framework

Step	Action	The Discipline
1. Reproduce	Confirm you can trigger the failure consistently. Three times.	A bug you can't reproduce, you can't verify as fixed. Never skip this.
2. Isolate	Narrow scope: UI layer? Service? Data? Network? Reduce to minimum inputs.	Binary elimination: does it fail on a clean install? With a test account?
3. Hypothesize	Form a specific, testable claim about the cause. Name the mechanism.	'Something is wrong' is not a hypothesis. Name the class and the mechanism.
4. Test	Make the smallest possible change to confirm or deny. One variable at a time.	If you make 3 changes and it works, you don't know which fixed it.
5. Fix	Address the root cause - not the symptom.	A nil check masks the cause. A non-optional design prevents it.
6. Verify	The bug is gone. No regressions. Tests pass. QA confirmed.	Not 'it runs.' Verified against every aspect of the original failure.
7. Test	Write a test that fails before the fix and passes after.	This step separates professional from amateur debugging.

Step 1: Reproduce the Bug

- A bug you cannot reproduce is a bug you cannot fix
- Before doing anything else, confirm that you can trigger the failure consistently
- Reproduction checklist:
  - What are the exact steps? Reproduce them three times to confirm consistency
  - What environment? Debug build or release?
  - Simulator or device?
  - Which OS version?
  - Which device model?
  - What data?
  - Was the user account in a specific state?
  - Are there specific input values that trigger it?
  - What is the frequency?
  - Does it happen every time, sometimes, or rarely?

If You Cannot Reproduce the Bug

- Pull the crash report from Crashlytics or Sentry
  - The stack trace tells you the exact code path, even for bugs you cannot reproduce manually
- Look for environmental dependencies:
  - Specific OS version
  - Specific locale
  - Specific network condition
  - Specific device capability
- If a user reported it: ask them to share their OS version, build number, and exact steps
  - Have them record a screen capture
- If it truly cannot be reproduced after exhaustive investigation:
  - Add diagnostic logging to the suspected code paths,
  - Release a debug build
  - Wait for the next occurrence to produce a log

Important!

Never close a bug as 'cannot reproduce' without documenting what you tried, what environment you tested in, and what the next steps would be if it resurfaces. 'Cannot reproduce' is a parking state, not a resolution.

Step 2: Isolate the Failure

- Narrow the scope of the problem before investigating code
  - The smaller the surface area of the failure, the less code you need to read
- Isolation strategies:
  - Layer isolation: Is the problem in the UI layer, service layer, data layer, or network layer? Test each independently. If calling the service method directly produces the wrong result, the UI is not the problem.
  - Input reduction: What is the minimum input that triggers the bug? Can you reduce a 50-field form to 2 fields that reproduce the failure? Minimum reproduction cases are far easier to debug than full application flows.
  - Binary elimination: Does the bug occur with a fresh install? With a clean test account? If yes, the problem is not in persisted state. If no, the problem may depend on accumulated state.
  - Environment elimination: Does it fail on iOS but not Android? On a real device but not a simulator? The difference points directly at the problem domain (platform API, hardware feature, build configuration).

Step 3: Form a Specific Hypothesis

- A hypothesis is a testable, specific claim about what is causing the bug
  - 'Something is wrong with the sync' is not a hypothesis
  - 'The SyncEngine marks entries as .synced before confirming the server returned HTTP 201, which means it marks them as synced even when the server is down' is a hypothesis.
- A good hypothesis:
  - Names a specific component or line of code
  - Describes a specific mechanism of failure (what is actually happening, not just that something is wrong)
  - Predicts a specific observable outcome (if this hypothesis is correct, then X should happen when I do Y)
- If you cannot form a specific hypothesis, you need more data
  - Add logging
  - Use the debugger to inspect state at runtime
  - Read the code more carefully
- Do not guess your way to a fix.

Step 4: Test the Hypothesis with Minimal Changes

- Test one hypothesis at a time. Change one variable at a time. If you make three changes and the bug disappears, you do not know which change fixed it
- Testing methods (from least to most invasive):
  - Add a log statement: confirm the code path is reached and what the values are at that point
  - Use the debugger: set a breakpoint and inspect state without changing any code
  - Add an assertion: XCTAssert or assert() to verify an invariant you believe should hold
  - Write a failing unit test: if your hypothesis is correct, this test should fail before you fix the bug
  - Make the smallest possible code change: if the test fails as predicted, make one targeted change and verify the test now passes

Step 5: Fix the Root Cause

- A fix that removes a symptom without addressing the root cause will recur - in the same place, or in a different place in the codebase where the same pattern exists. The fix must address why the bug occurred, not just prevent the specific manifestation.

Symptom Fix (Wrong)	Root Cause Fix (Right)	Why the Difference Matters
Wrap the force-cast in a try-catch to avoid the crash	Replace all force-casts with safe optional binding; add a linting rule that prevents force-casts in notification handlers	The try-catch hides the bug - the next notification with a missing field still arrives with no error. The root cause fix prevents the entire class of error.
Add a nil check before the crashing line	Understand why nil is possible here and either enforce non-nil at the call site or design the API to return a non-optional type when appropriate	The nil check is reactive. The non-optional design is proactive.
Reload the data after the crash by catching the error and retrying	Fix the state transition logic that produces an invalid state the UI cannot render	Retry masks the invalid state. The state will become invalid again.

Step 6: Verify the Fix

- Verification is not 'run the app and it works'
- Verification is a systematic check that the fix resolves the bug and does not introduce new bugs
- Verification steps:
  - Reproduce the original bug with the fix applied. Confirm it is resolved
  - Run all existing tests. A fix that breaks other tests has introduced a regression
  - Test adjacent behaviors: if you changed how errors are handled in one code path, test other error paths in the same module
  - If the bug occurred in production and a QA environment is available, test in QA before shipping
  - If possible, have another engineer verify the fix on a different device or configuration

Step 7: Write a Test That Would Have Caught the Bug

- This step is what separates professional debugging from amateur debugging
- Every bug that reaches production is evidence of a gap in the test suite
- Filling that gap ensures the bug cannot regress undetected
- The test has three properties:
  - It fails before the fix is applied (it reproduces the bug)
  - It passes after the fix is applied (it verifies the fix)
  - It will catch a regression if the bug is accidentally reintroduced in the future

Note

Teams that consistently follow Step 7 accumulate a regression test suite that grows with every bug found in production. Over time, the most common categories of defect become impossible to ship - they are caught by the test suite before they leave a developer's machine.

Knowledge Check

Knowledge Check · Question 1

A developer estimates 3 points. A senior engineer estimates 13. After brief discussion, the team compromises on 5 without the senior fully explaining their reasoning. What went wrong?

Knowledge Check · Question 2

What distinguishes a good root cause (the final answer in a Five Whys) from a poor one? Give an example of each.

Knowledge Check · Question 3

A crash report points to: let weekSummary = timesheets.first!.weekSummary. Apply the full 7-step debugging framework.

Knowledge Check · Question 4

What is a technical spike? When is one required? What must its output include?

Knowledge Check · Question 5

What does 'error ownership' mean in a mobile architecture? Why is 'no layer swallows an error silently' a critical rule?

PHASE 01 COMPLETE

What You Can Now Do

Phase 01 - What You Can Do After This Week

- Describe and apply the SDLC, run a Scrum sprint, and write INVEST-compliant user stories
- Evaluate mobile platform tradeoffs and select the right approach for a given project
- Apply all five SOLID principles to audit and refactor code
- You can use Git confidently - branching, rebasing, conflicts, advanced operations
- You can apply a branching strategy and enforce it through CI and pull request discipline
- You can decompose ambiguous requirements into executable, dependency-ordered tasks
- You can debug systematically using platform tools - not randomly
- You can trace a production defect to its systemic root cause
- You can produce an architecture description with data flow, failure modes, and error ownership

Next: Phase 02

iOS Development with Swift · Weeks 2-4